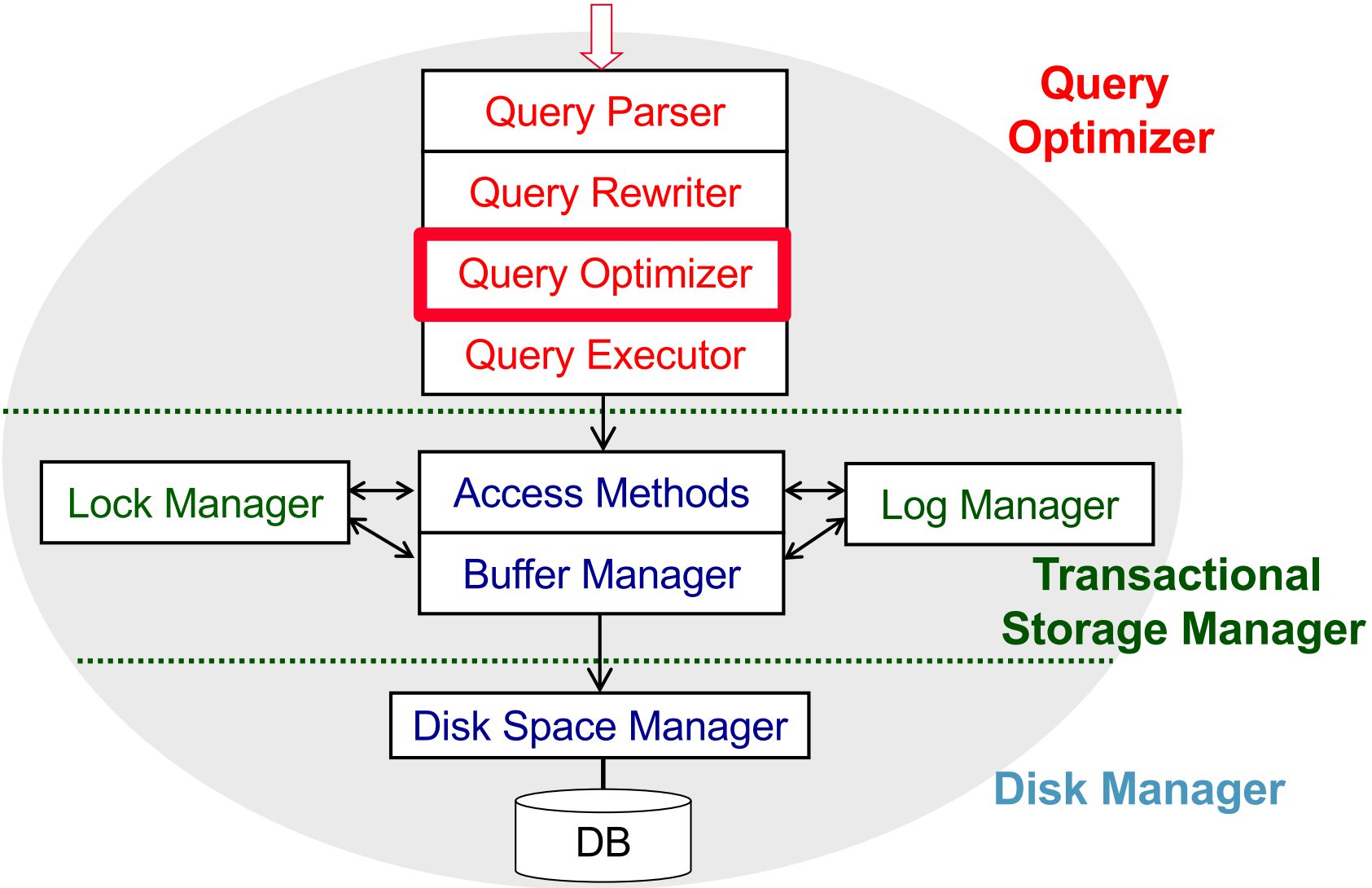
Relational Query Optimization

Ioana Manolescu
Inria and Ecole Polytechnique



Slide Content Courtesy of R. Ramakrishnan, J. Gehrke, J. Hellerstein and Y. Diao

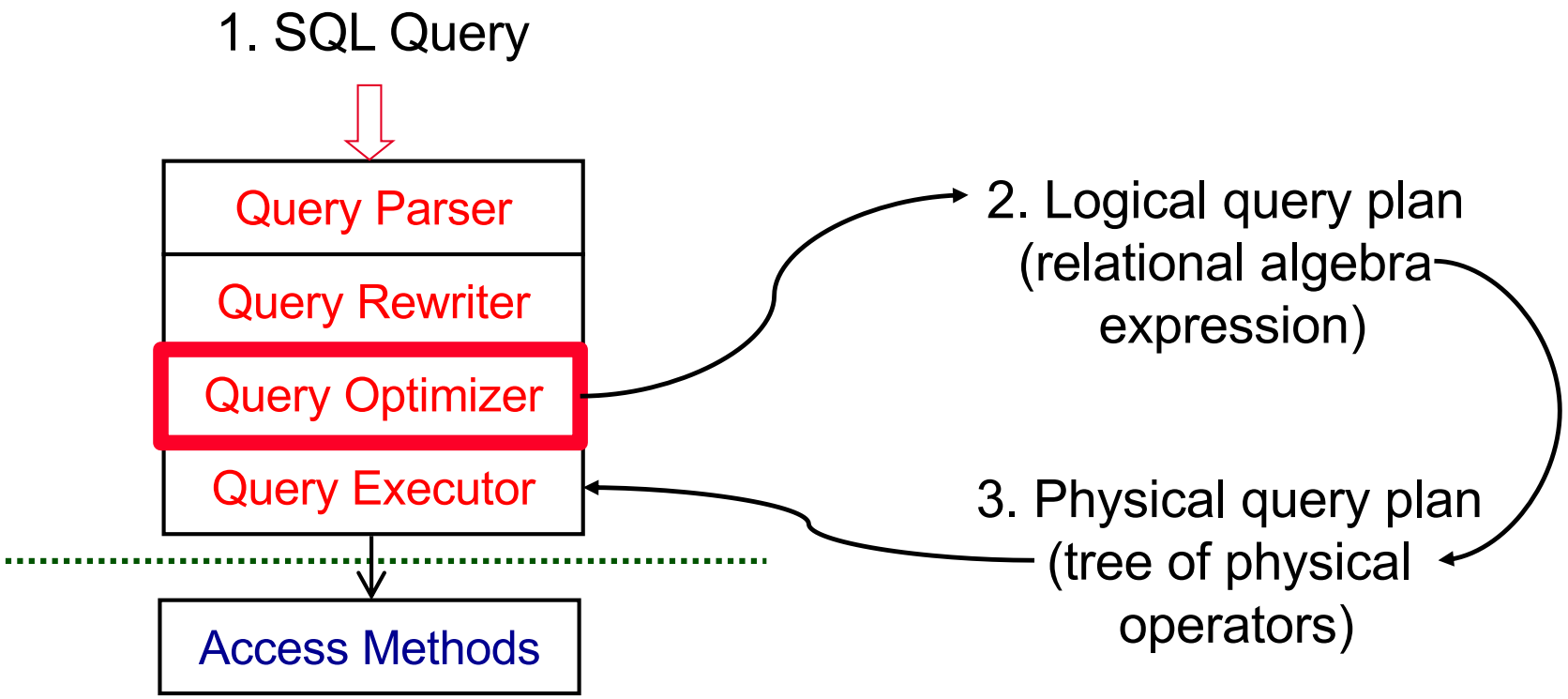
Overview of Query Processing



Outline

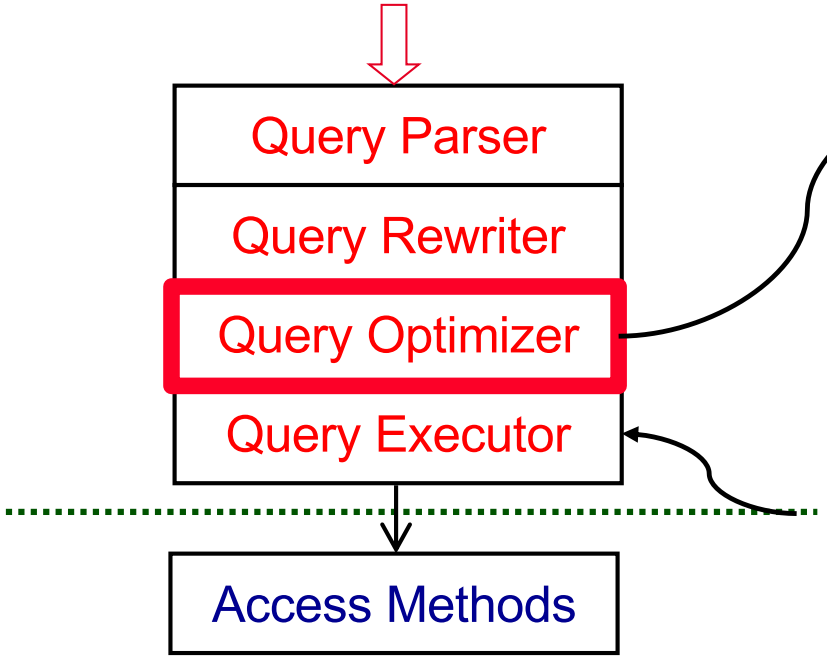
- Overview of query optimization
- Enumeration of logical plans
- Enumeration of physical plans
- Estimating plan costs
- Search strategy
- Optimization of nested queries

Simple overview of query processing



Simple overview of query processing

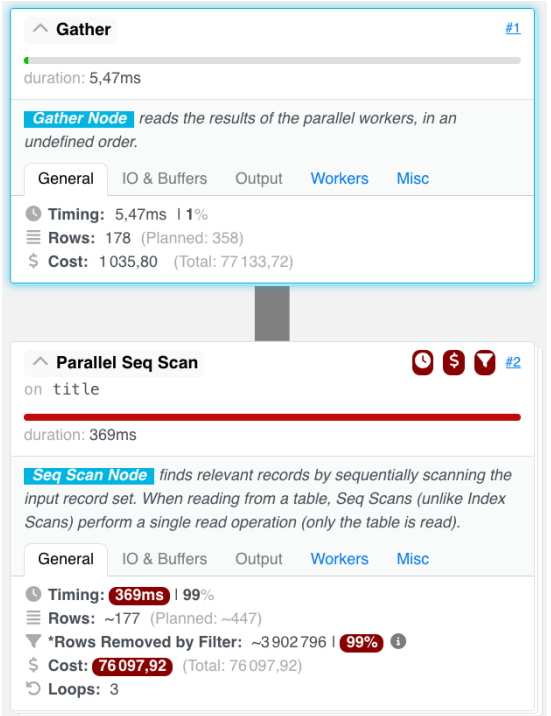
1. SQL Query: select title, production_year
from title where title like 'Godzilla%'

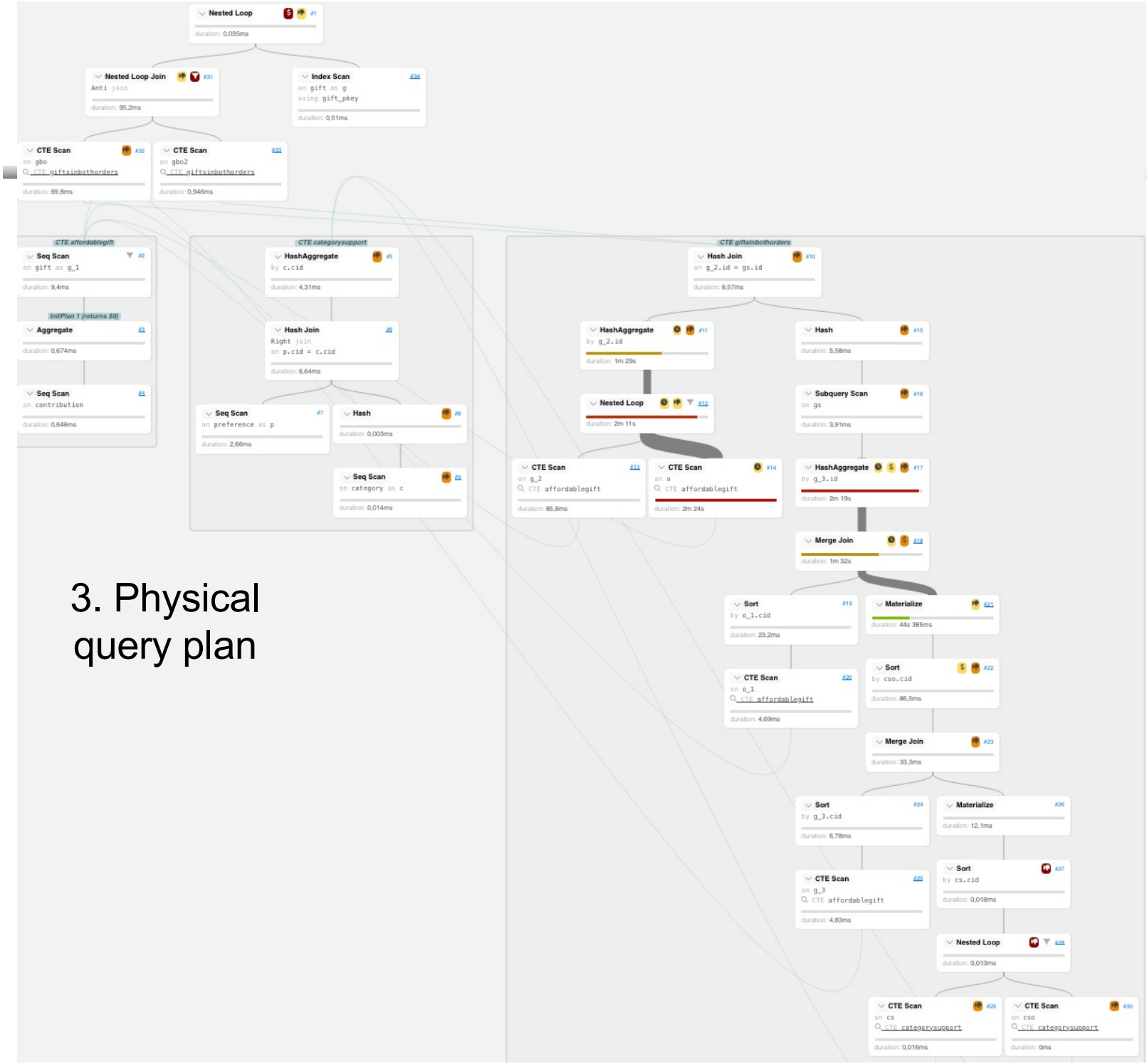


2. Logical query plan

$$\pi_{title,prod_{year}}(\sigma_{title \geq 'Godzilla' \wedge title \leq 'Godzillb'}(title))$$

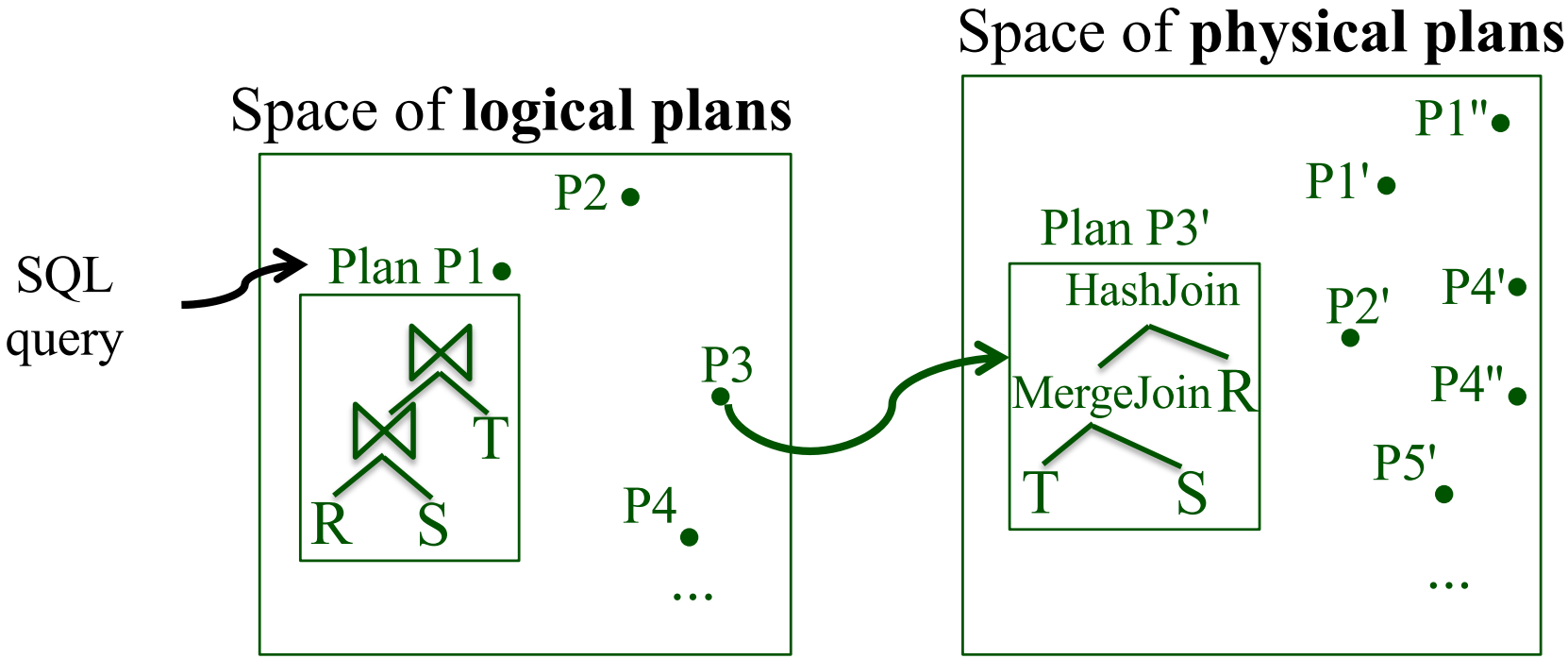
3. Physical query plan





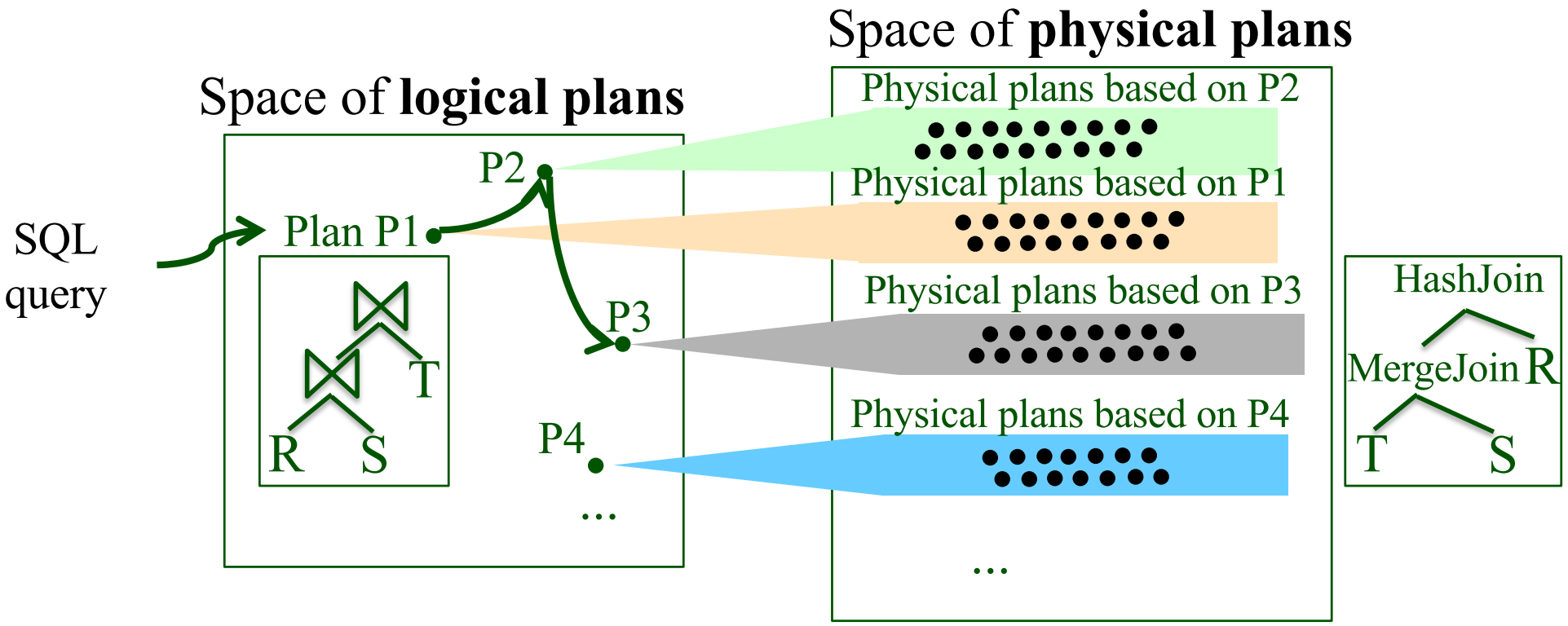
3. Physical query plan

Detailed query optimization process



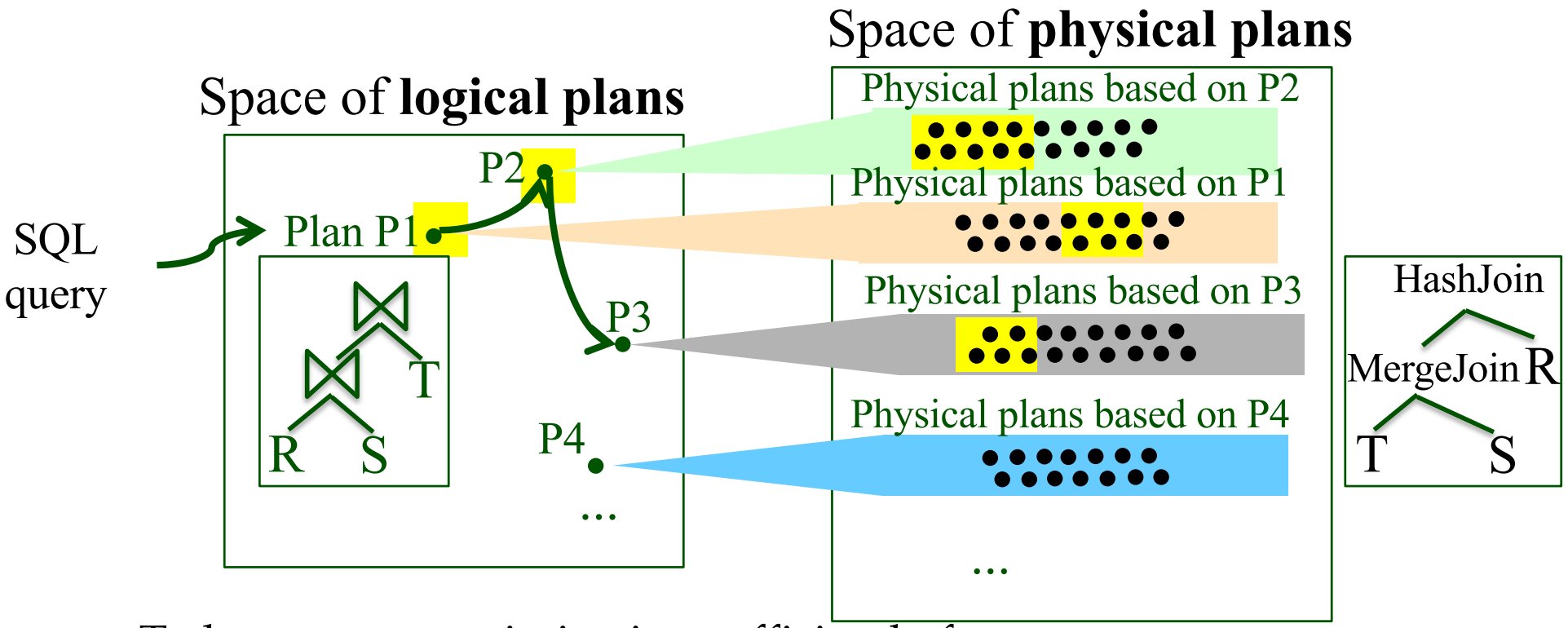
- For a query, there are usually **several, equivalent logical plans**
- To each logical plan correspond multiple **physical plans** (aka evaluation plans)
- Every physical plan has a **cost**

Detailed query optimization process



- To each logical plan corresponds a set of physical plans
  - To a logical plan we can "associate a cost" which is the *lowest* cost of a physical plan implementing it

Detailed query optimization process



- To keep query optimization sufficiently fast, only *some* logical plans are explored
  - Only *some* of the corresponding physical plans are explored
  - Impact on the "cost" of a logical plan
- Many *heuristics*

Three main issues in optimization

- Given a query block, three main optimization issues:

1. *Plan space*: what plans are considered?

1. *Plan cost*: what is the cost of each plan?

1. *Search algorithm*: how do we search the plan space for the cheapest estimated plan?

Unit of optimization: query block

- An SQL query is parsed into a collection of *query blocks*, and these are optimized one block at a time.

```
SELECT  S.sname
FROM    Sailors S
WHERE   S.age IN
        (SELECT  MAX (S2.age)
         FROM  Sailors S2
         GROUP BY  S2.rating)
```

Outer block *Nested block*

- How to handle nested blocks?
  - Remove them (**unnest** the query) if possible
  - Otherwise, nested blocks are usually treated as calls to a subroutine, made **once per tuple of the outer block**, or **materialized** and used like another table

Outline

- Overview of query optimization
- Enumeration of logical plans
- Enumeration of physical plans
- Estimating plan costs
- Search strategy
- Optimization of nested queries

Schema for examples

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)
Reserves (*sid*: integer, *bid*: integer, *day*: dates, *rname*: string)

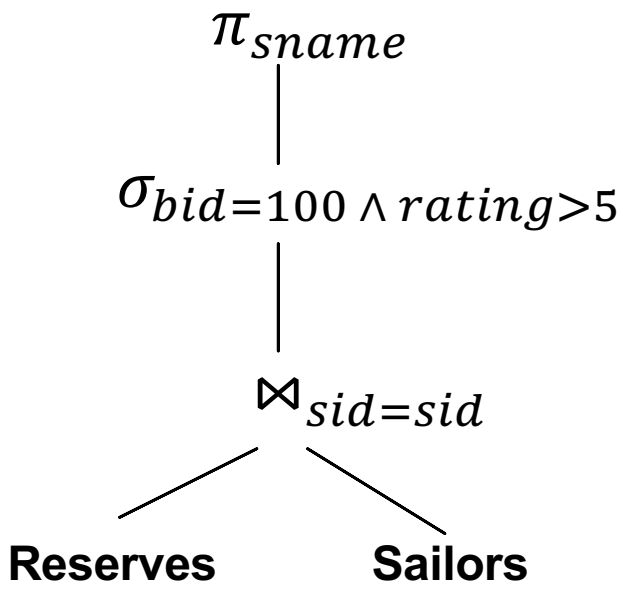
- Reserves:
  - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages.
- Sailors:
  - Each tuple is 50 bytes long, 80 tuples per page, 500 pages.

Logical plan

Aka relational algebra expression

```
SELECT  S.sname
FROM    Reserves R, Sailors S
WHERE   R.sid=S.sid AND
        R.bid=100 AND S.rating>5
```

Tree form:



Linear form:

$\pi_{sname}\ (\sigma_{bid=100 \wedge rating>5}\ (Reserves \bowtie_{sid=sid} Sailors))$

Main stages of logical plan construction

```
SELECT      {DISTINCT} <list of columns>
FROM        <list of relations>
{WHERE      <list of conditions>}
{GROUP BY <list of columns>
{HAVING     <list of conditions>}}
{ORDER BY <list of columns>};
```

- Query Semantics:
  1. Take Cartesian product (a.k.a. cross-product) of relns in FROM, projecting only to those columns that appear in other clauses
  2. If a WHERE clause exists, apply all filters in it
  3. If a GROUP BY clause exists, form groups on the result
  4. If a HAVING clause exists, filter groups with it
  5. If an ORDER BY clause exists, make sure output is in right order
  6. If there is a DISTINCT modifier, remove duplicates

Main stages of logical plan construction

```
SELECT      {DISTINCT} <list of columns>
FROM        <list of relations>
{WHERE      <list of "Boolean Factors">}
{GROUP BY <list of columns>
   {HAVING     <list of Boolean Factors>}}
{ORDER BY <list of columns>};
```

1. Convert selection conditions to a normal form, e.g., *conjunctive normal form* (CNF):
   - (*day<8/9/94 OR bid=5 OR sid=3*) *AND* (*rname= 'Paul' OR sid=3*)

2. Interleave FROM and WHERE into an *operator tree for optimization*.

3. Apply GROUP BY, HAVING, DISTINCT and ORDER BY at the end, pretty much in that order.

From this "blueprint", many alternatives through *equivalences*

Basic Relational Algebra Equivalences

- *Selections*: $\sigma_{c1 \wedge \ldots \wedge cn}(R) \equiv \sigma_{c1}(\ldots \sigma_{cn}(R))$ *(Cascade)*

  $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$ *(Commute)*

- *Projections*:

  $\pi_{a1}(R) \equiv \pi_{a1}(\ldots(\pi_{an}(R)))$ *(Cascade)*

- *Joins*: $(R \bowtie S) \equiv (S \bowtie R)$ *(Commute)*

  $R \bowtie (S \bowtie T) \equiv (R \bowtie S) \bowtie T$ *(Associative)*

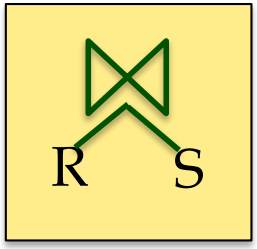
More Equivalences

- $\sigma_c (R \times S) \equiv R \bowtie_c S$
- $\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S$ if c only involves attributes from R
- $\pi_a(\sigma_c(R)) \equiv \sigma_c(\pi_a(R))$ if σ only uses attributes retained by π
- For $\pi_b(R \bowtie_a S)$, we can 'push' π before $\bowtie$ by retaining only the *a* attributes and the *b* attributes

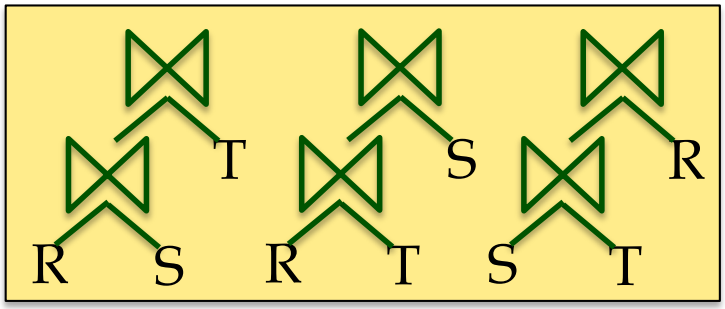
Plan space: join ordering

- *Join orders* = all trees having the relations as leaves

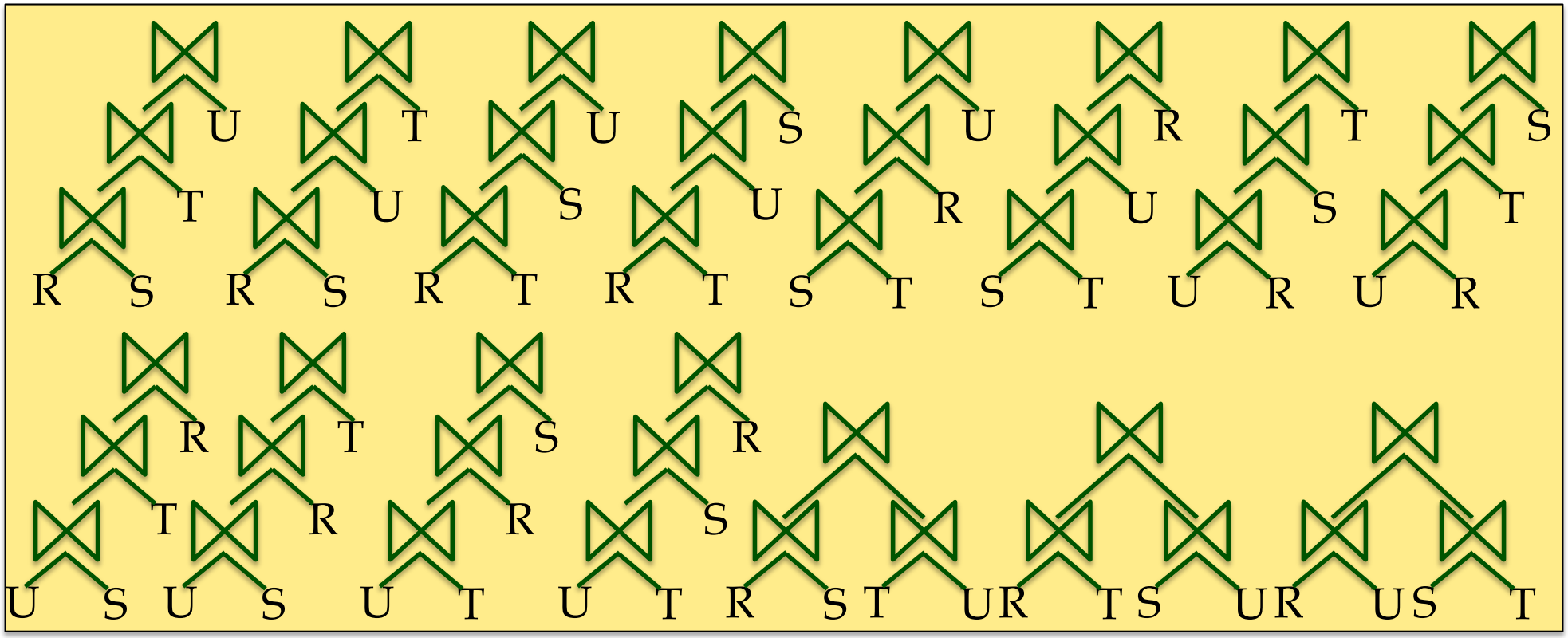
N=2:



N=3:

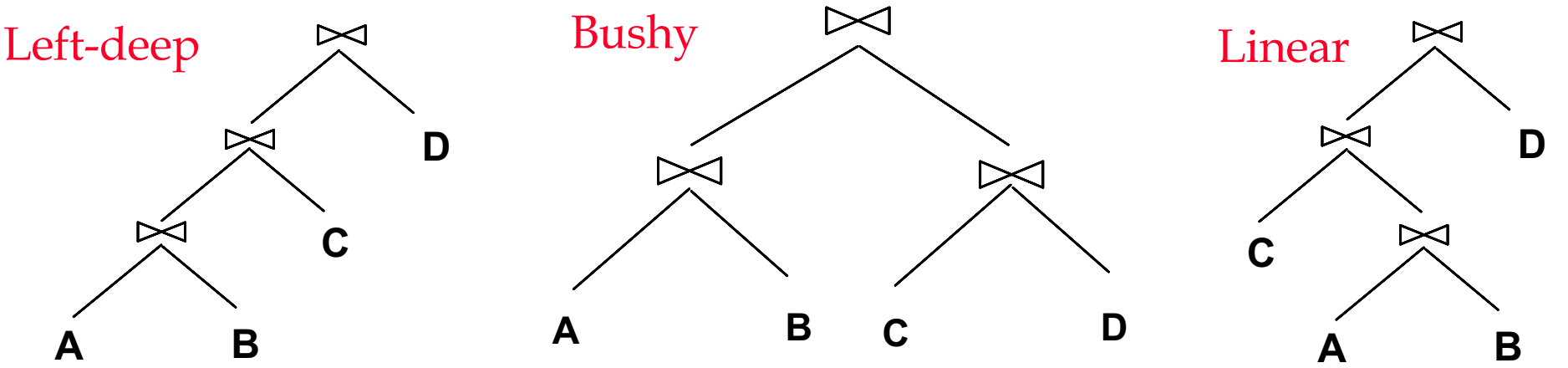


N=4:



The System R Optimizer

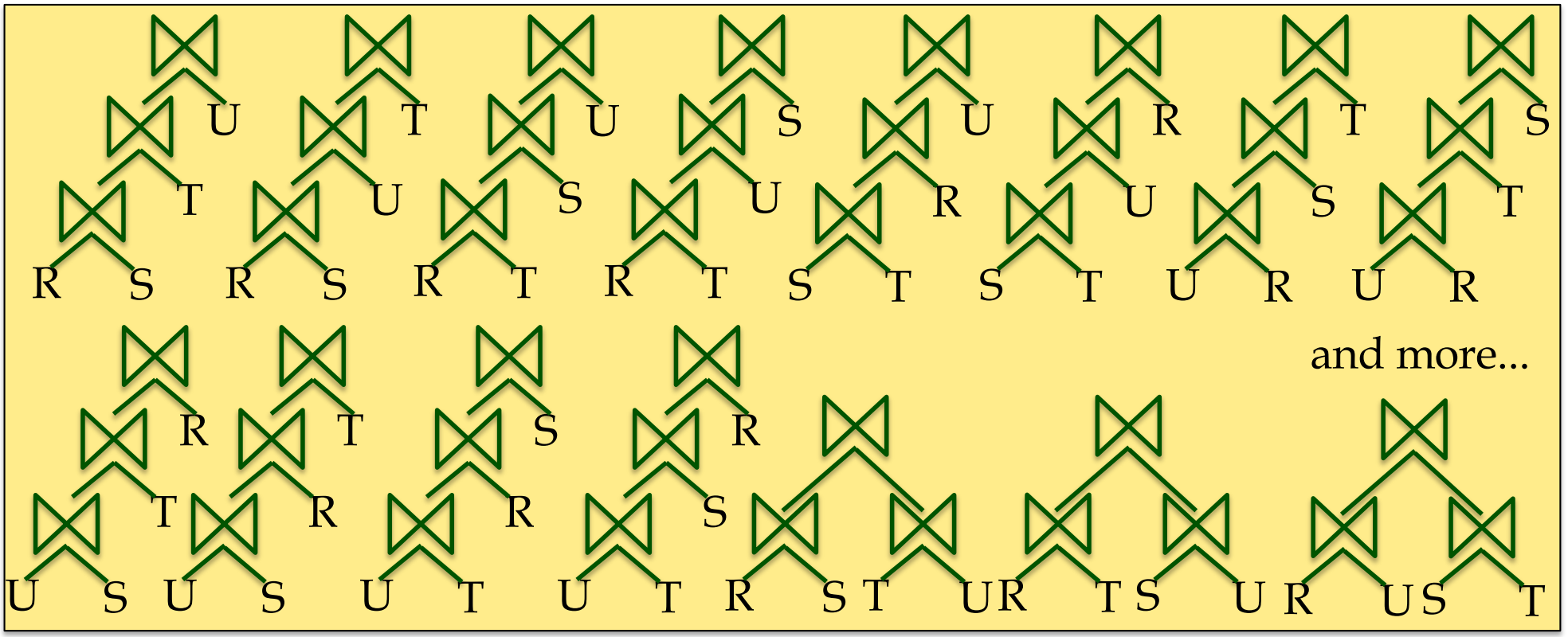
- IBM San Jose, late 1970
- Most current RDBMS optimizers somehow derive from it
- Plan Space: too large, must be *pruned*.
  - Avoids Cartesian products
  - Only considers *left-deep plans* (inner is always a relation): n!
  - Uses *linear plans* (inner or outer is a relation) and ignores *bushy*



Plan space: join ordering

- *Join orders* = all trees which can be built with relns as leaves
- Number of bushy plans?

$$BP(n+1) = \sum_{k=1}^{n} C_{n+1}^{k} * BP(k) * BP(n+1-k)$$



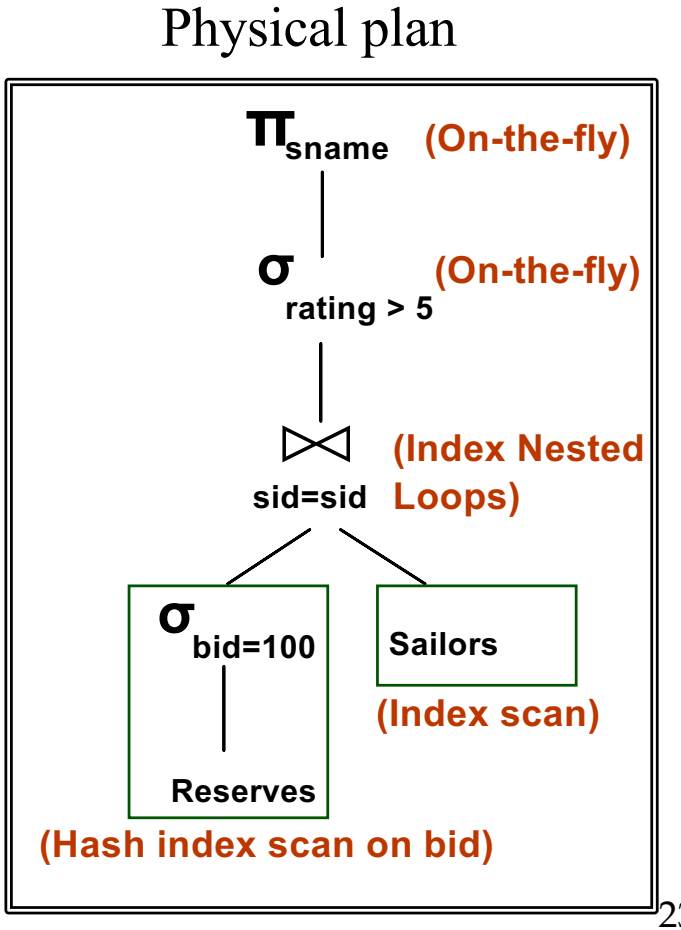
Outline

- Overview of query optimization
- Enumeration of logical plans
- Enumeration of physical plans
- Estimating plan costs
- Search strategy
- Optimization of nested queries

Inside the executor

- A physical plan is a tree composed of physical operators
- How do they synchronize?

- Two main options
  - **Pull-based** (iterators)
    - Most frequently used in RDBMSs
    - Operators self-synchronize
  - **Push-based**
    - Fashionable in the last decade
    - Less synchronization effort
    - Better cache locality



Operator synchronization: iterators

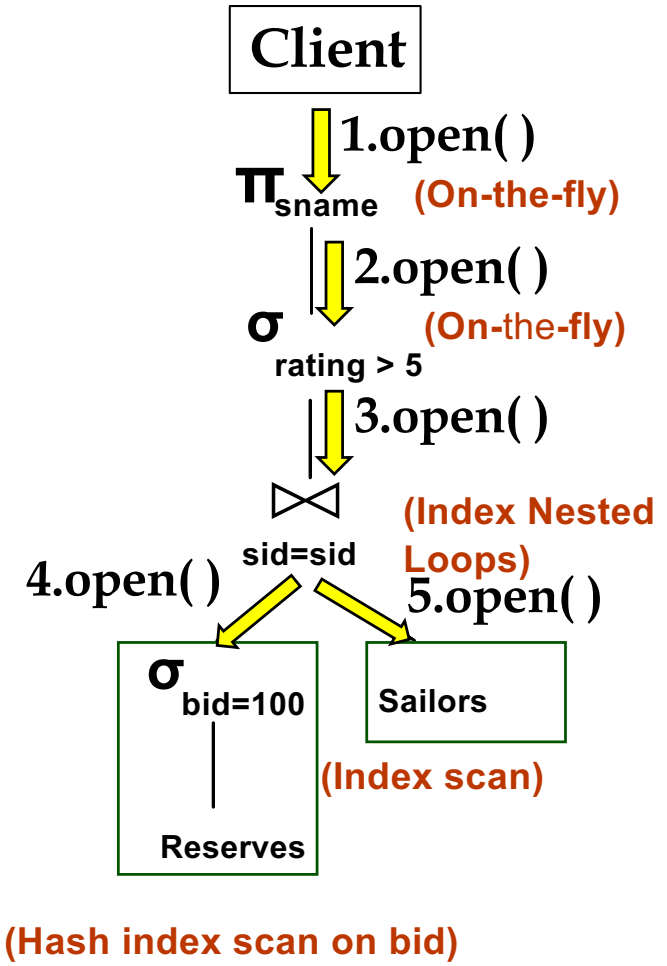
Popular *Iterator Interface* implemented by each physical operator:

open, *get_next*, and *close*.

open allocates resources (memory...) and performs any other initialization

get_next returns the next result (if any)

close releases resources



Operator synchronization: iterators

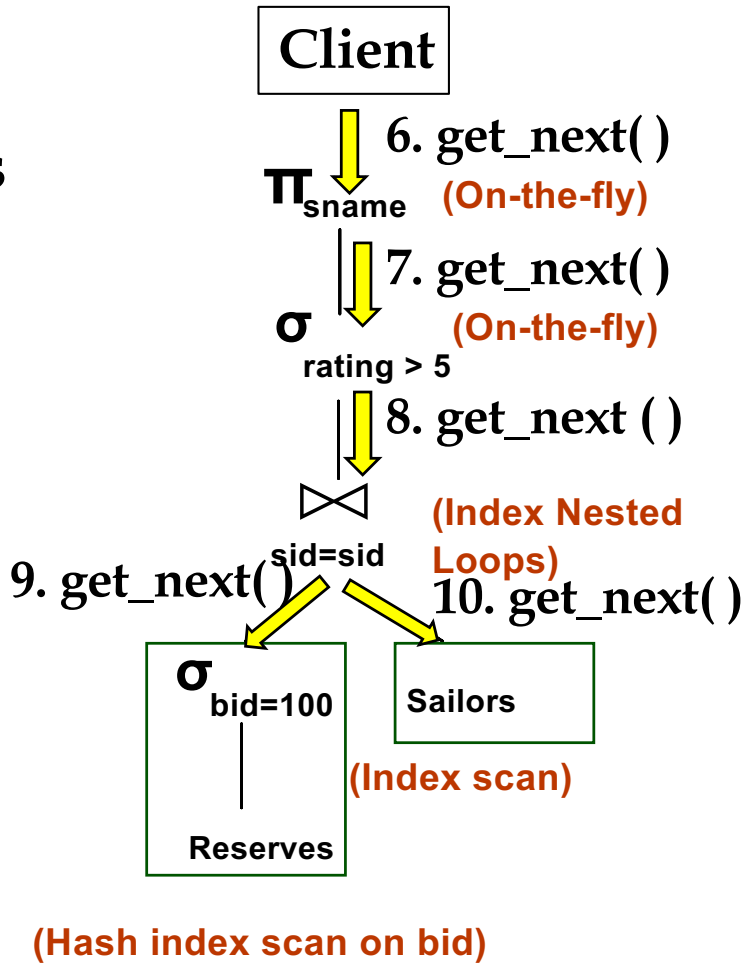
Popular *Iterator Interface* implemented by each physical operator:

open, *get_next*, and *close*.

open allocates resources (memory...) and performs any other initialization

get_next returns the next result (if any)

close releases resources



Operator synchronization: iterators

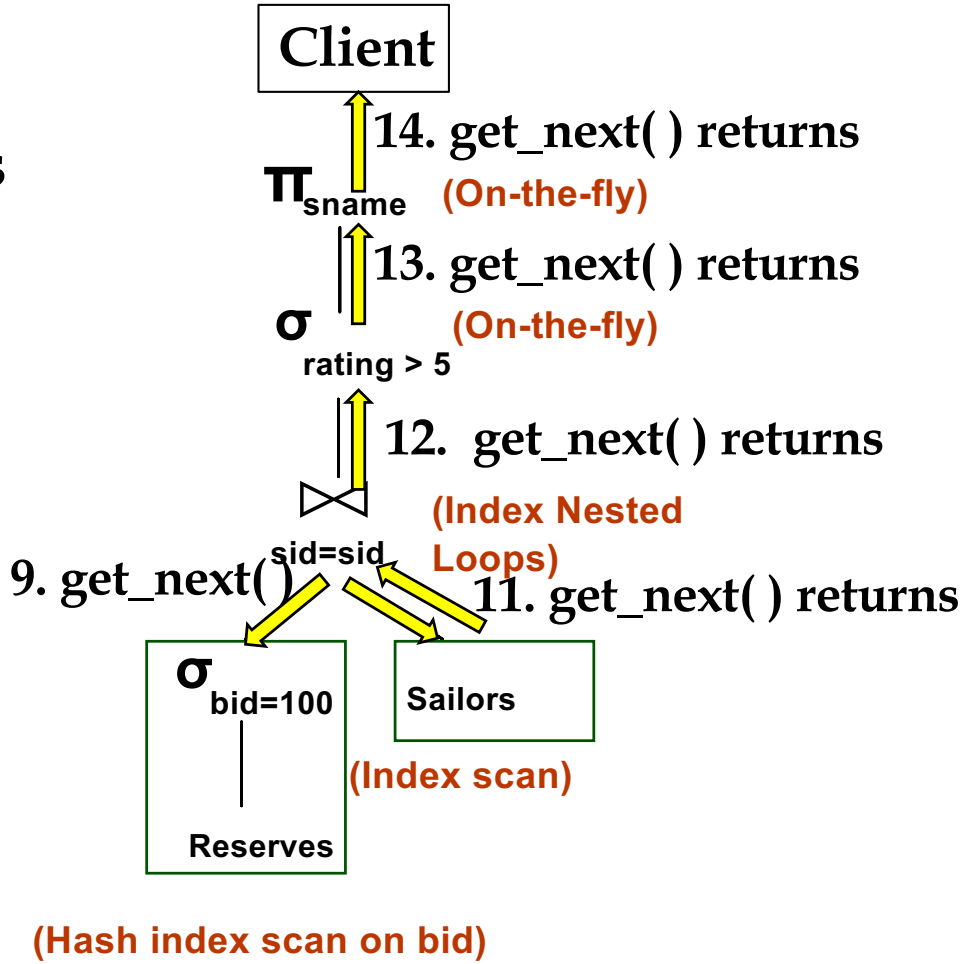
Popular *Iterator Interface* implemented by each physical operator:

open, *get_next*, and *close*.

open allocates resources (memory...) and performs any other initialization

get_next returns the next result (if any)

close releases resources



Operator synchronization: iterators

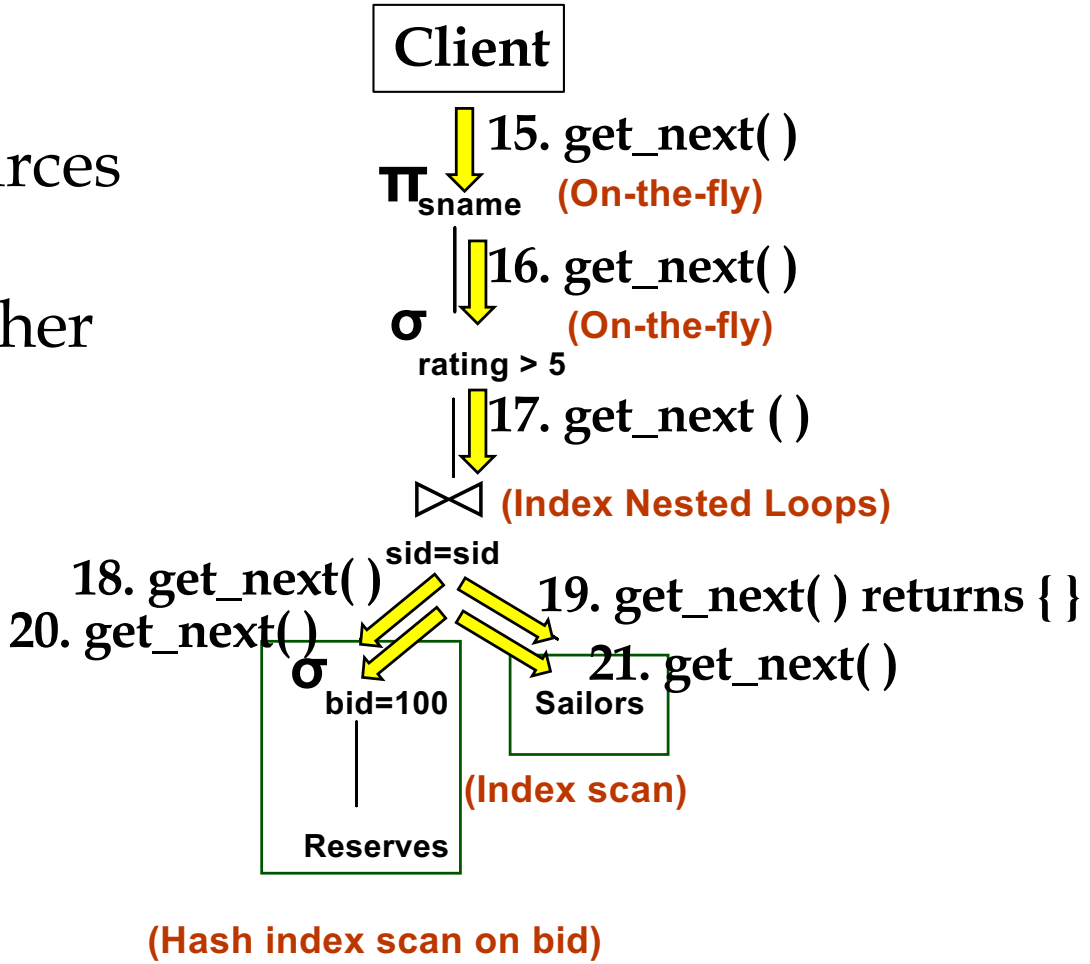
Popular *Iterator Interface* implemented by each physical operator:

open, *get_next*, and *close*.

open allocates resources (memory...) and performs any other initialization

get_next returns the next result (if any)

close releases resources



Operator synchronization: iterators

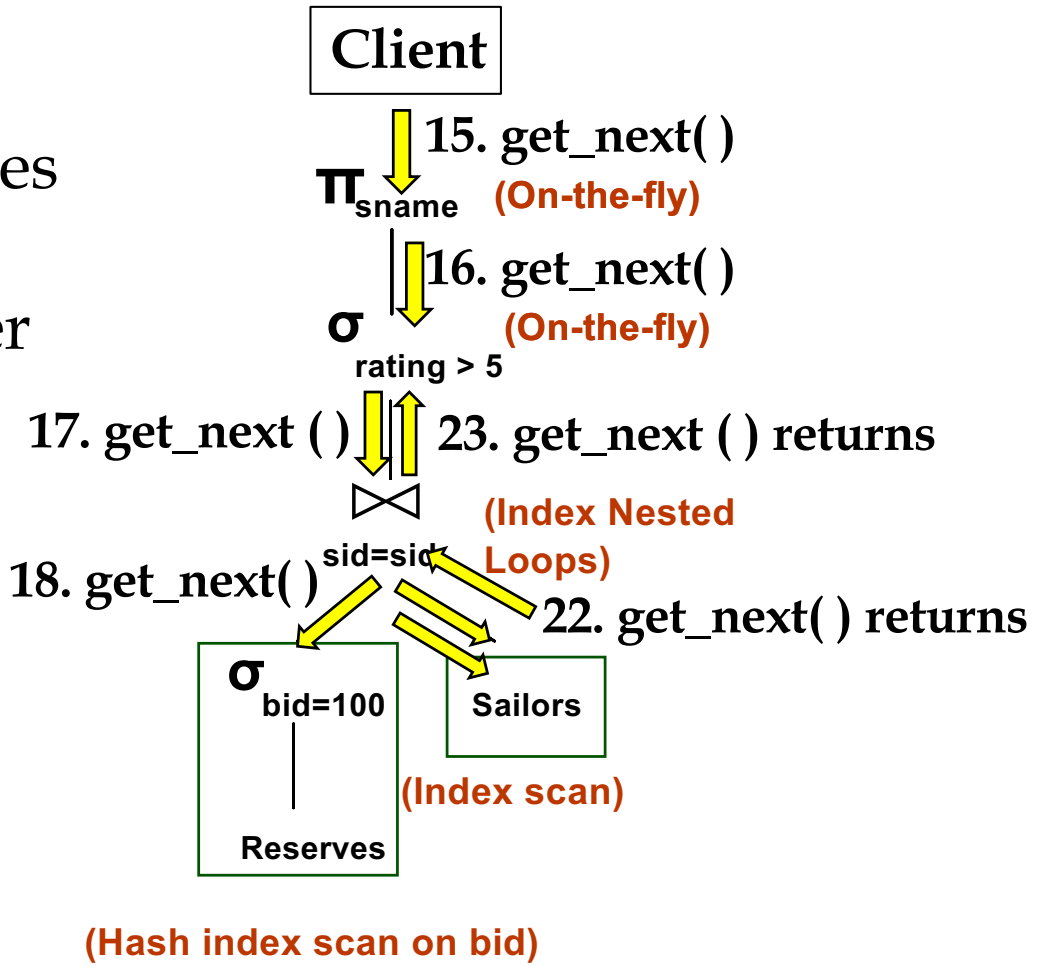
Popular *Iterator Interface* implemented by each physical operator:

open, *get_next*, and *close*.

open allocates resources (memory...) and performs any other initialization

get_next returns the next result (if any)

close releases resources



Operator synchronization: iterators

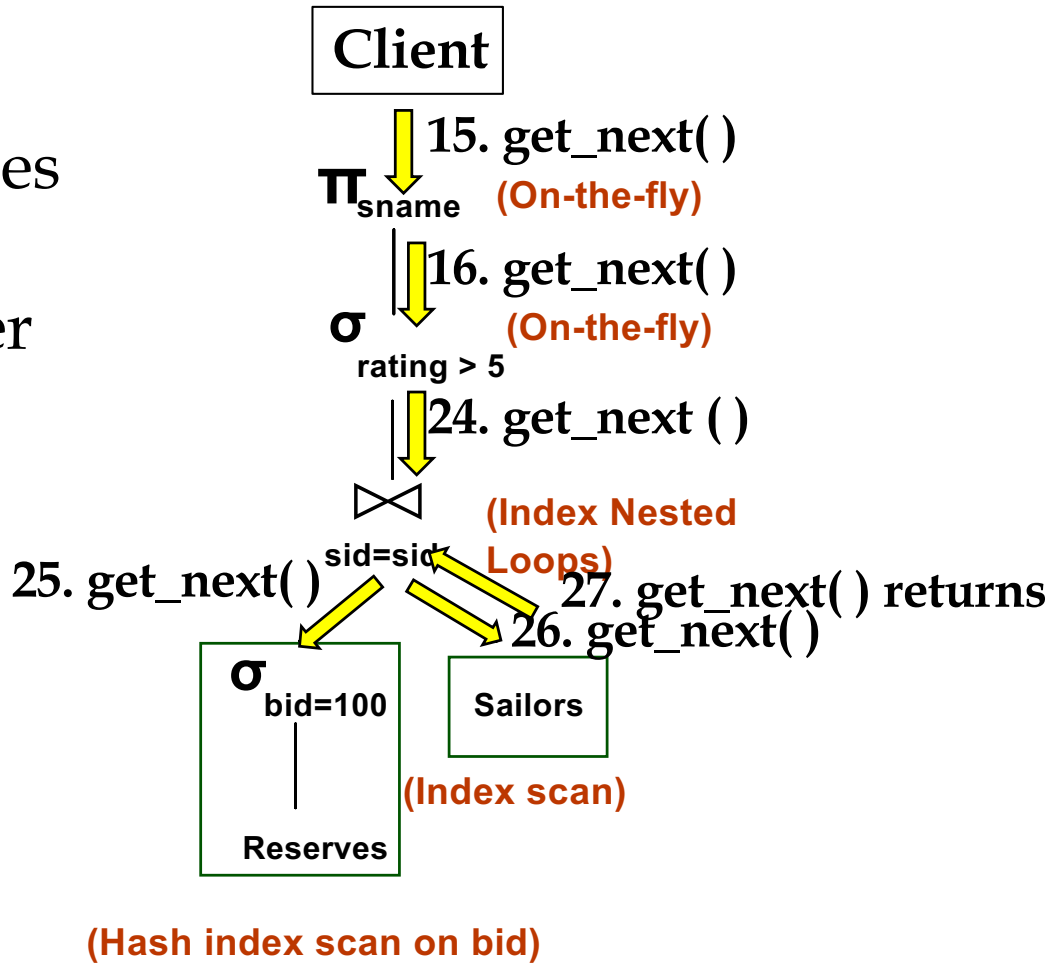
Popular *Iterator Interface* implemented by each physical operator:

open, *get_next*, and *close*.

open allocates resources (memory...) and performs any other initialization

get_next returns the next result (if any)

close releases resources



Operator synchronization: iterators

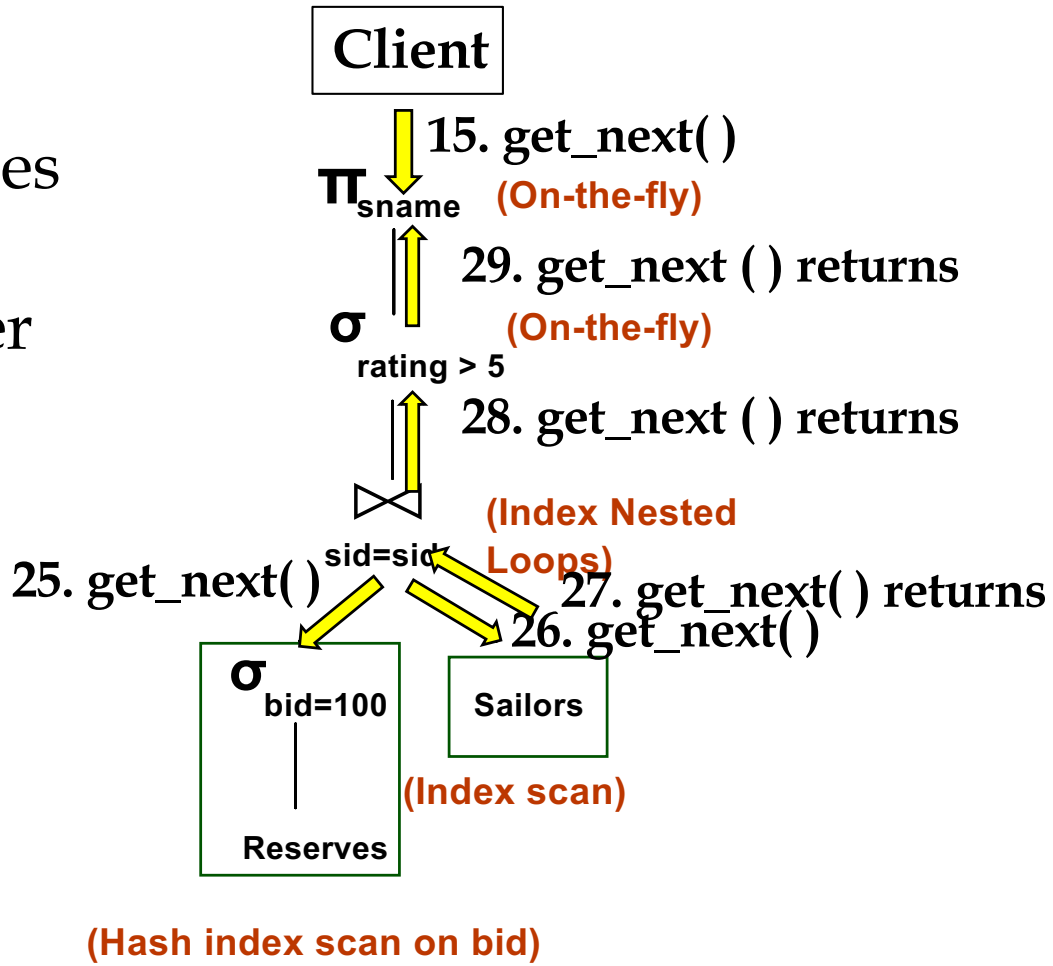
Popular *Iterator Interface* implemented by each physical operator:

open, *get_next*, and *close*.

open allocates resources (memory...) and performs any other initialization

get_next returns the next result (if any)

close releases resources

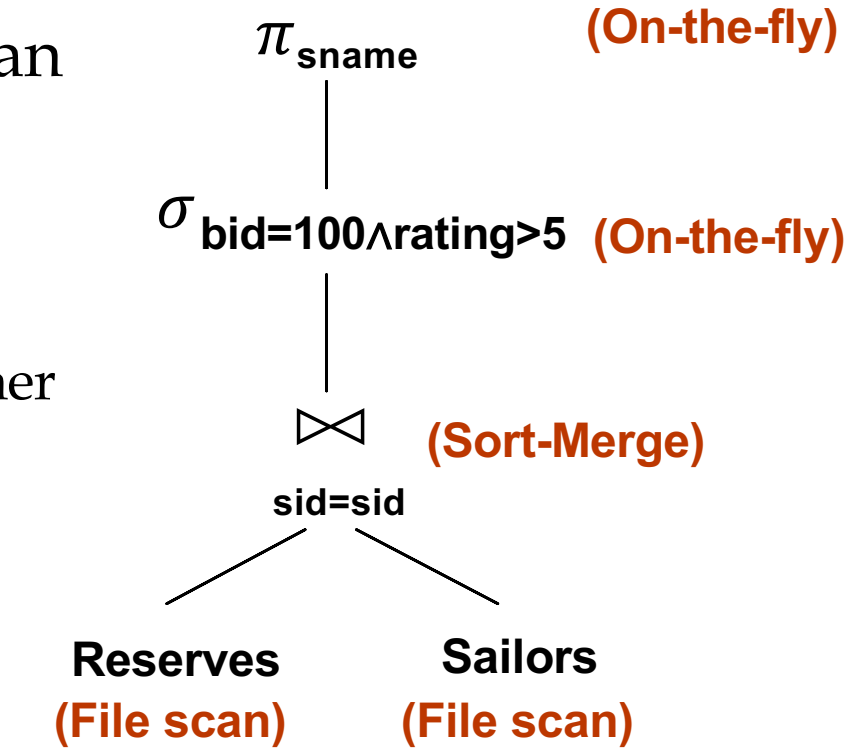


Iterators

- Advantages
  - Encapsulate operator algorithm (logic)
    - All operators behave the same from the outside, regardless of how they are implemented
  - Self-synchronizing
    - Pull-based
- Shortcomings
  - One method call for every tuple and every operator that manipulates it
  - In *memory-resident databases*, next( ) calls have poor cache locality
  - These shortcomings are addressed by push-based methods.

Query evaluation plan

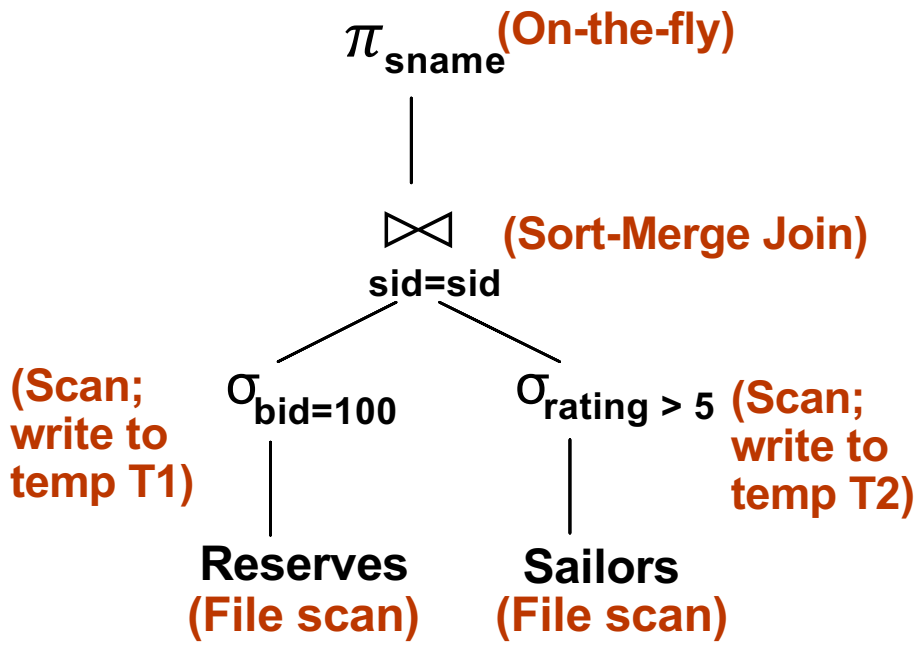
- *Query evaluation plan* extends an RA tree with implementation for each operator:
  - *access method* for leaves;
  - *implementation method* for the other operators.



- Missed opportunities in this plan:
  - Selections could have been 'pushed' earlier.
  - More efficient joins.
  - Use of indexes.

Query Plan 1 (Selection Pushed Down)

- *Push selections below the join.*
- *How to pass output of σ to the next operators?*
  - *Materialization* vs. *Pipelining*

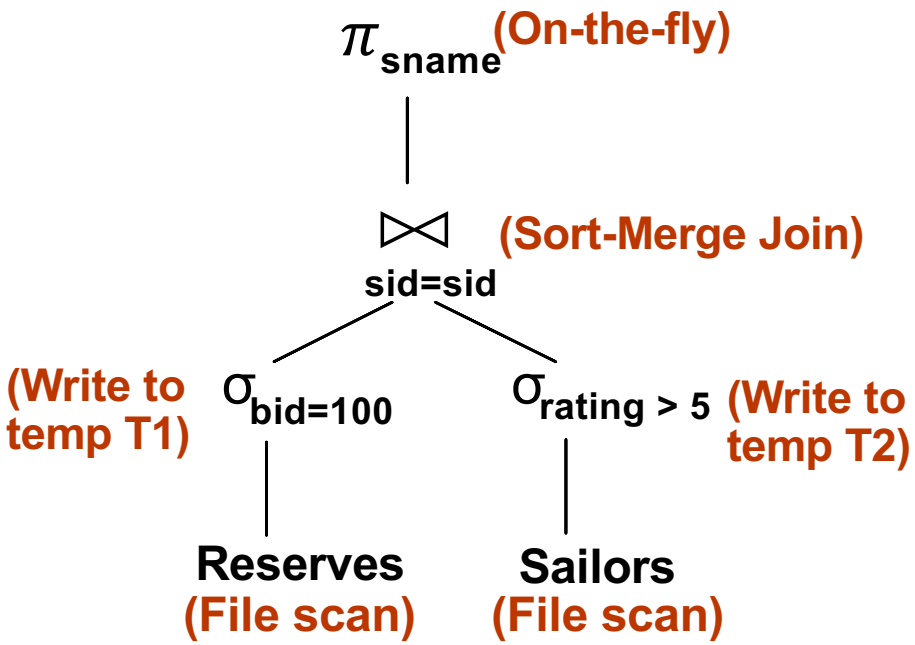


Materialization vs. Pipelining

- *Materialization*: Output of an *op* is saved in a temporary relation.
- *Pipelining*: No need to create a temporary relation. Avoid the cost of writing it out and reading it back. Can occur in two cases:
  - *Unary operator*: when the input is pipelined into it, the operator is applied *on-the-fly*, e.g. selection on-the-fly, project on-the-fly.
  - *Binary operator*: e.g., the outer relation in indexed nested loops join.

Query Plan 1 (Selection Pushed Down)

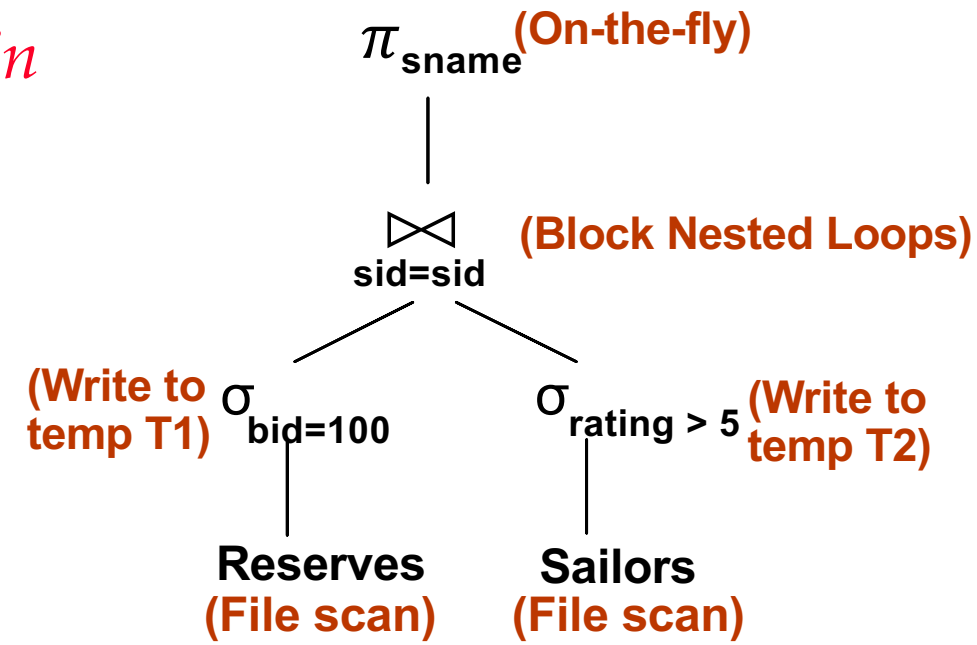
- *Push selections below the join*



- With 5 buffer pages (B=5), cost of plan:
  - Scan Reserves (1000 pages) + write temp T1 (10, assume matching boats fit in 10 pages).
  - Scan Sailors (500) + write temp T2 (assume 250 pages).
  - Sort-Merge join: Sort T1 (2*2*10), sort T2 (2*4*250), merge (10+250).
  - Total = 4060 page I/Os.

Query Plan 2 (Different Join Method)

- Join uses *block nested loops join* instead.



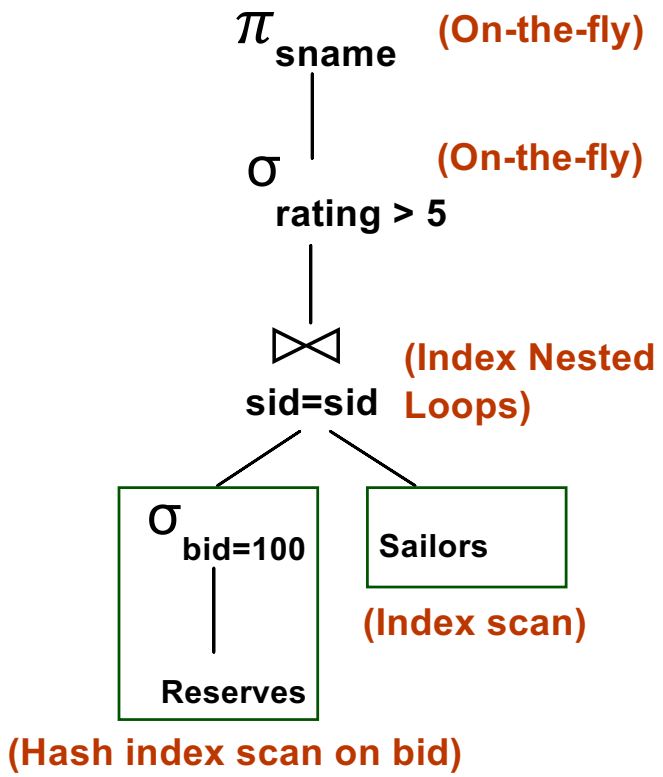
- With 5 buffer pages (B=5), cost of plan:
  - Scan Reserves (1000) + write temp T1 (10 pages).
  - Scan Sailors (500) + write temp T2 (250 pages).
  - BNL join: "block" size for reading R is 5-2=3 (1 for S, 1 for output); we need 4 passes to read R; join cost = 10+4*250.
  - Total cost = 2770.

Chosing the evaluation method for plan leaves: when to use an index

- A **tree** index *matches* (a conjunction of) terms if the attributes in the terms form a *prefix* of the index key.
  - Tree index on <*a, b, c*>
  - *a=5 AND b=3* ?
  - *a=5 AND b>6 ?*
  - *b=3 ?*
- A **hash** index *matches* (a conjunction of) terms if there is a term *attribute* = *value* for *every* attribute in the index key.
  - Hash index on <*a, b, c*>
  - *a=5 AND b=3 AND c=5*
  - *a=5 AND b=3*
  - *a>5 AND b=3 AND c=5*

Query Plan 3 (Using Indexes)

- *Selection using index*: clustered index on *bid* of Reserves
  - Retrieve 100,000/100 = 1000 tuples
  - Clustering: read 1000/100 = 10 pages.
- Indexed NLJ: *pipeline* the outer and *index lookup* on sid of Sailors.
  - The outer: no need to materialize.
  - The inner: *sid* is a *key*; *at most one* match tuple, unclustered index OK.



- Cost:
  - Selection of Reserves tuples (10 I/Os).
  - For each tuple, get matching Sailors tuple (1000*(1.2+1)).
  - Total = 2210 I/Os.

Recap: Query Plan Costs

	Plan 1	Plan 2	Plan 3
Access to R (also the σ)	1000+10	1010	10
Access to S	500+250 (also the σ)	750 (also the σ)	2200 (index nested-loops join) σ on the fly after the join
Joining R and S	Sort: 40+2000 Merge: 260	1010 (block nested-loops join)	
Total	4060	2770	2210
	Poor man's choice (5 buffer pages)	**Smarter option even with little memory**	**"Deluxe" option (2 indexes, query well served). Frequent in well-designed settings!**

Outline

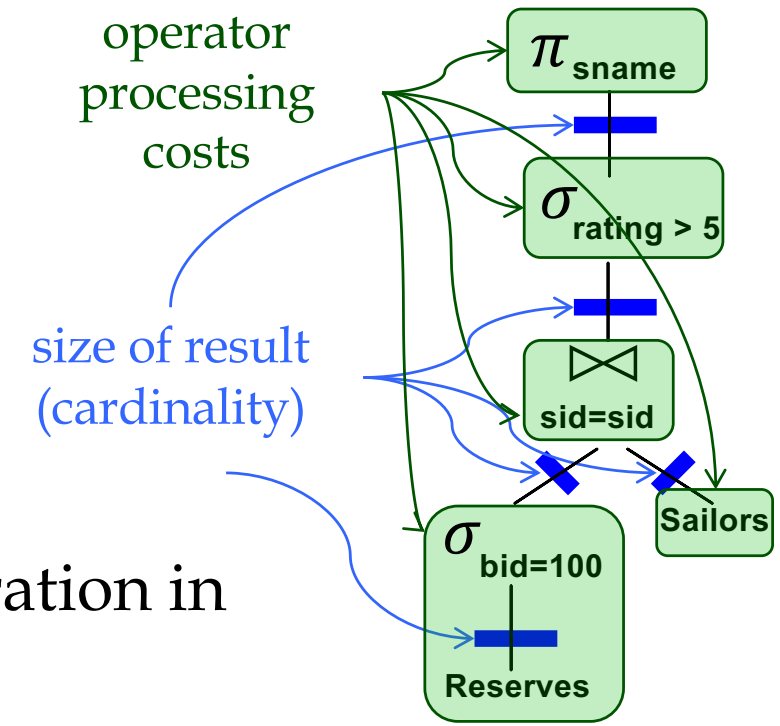
- Overview of query optimization
- Enumeration of logical plans
- Enumeration of physical plans
- Estimating plan costs
- Search strategy
- Optimization of nested queries

Estimating plan costs

- On physical plans

- Estimate *cost* of each operator:
  - Depends on input size.
  - Depends on the method (sequential scan, index scan, join method...)

- Estimate *size of result* for each operation in tree:
  - Use statistics about input relations.



Statistics in system catalog

- Statistics about each relation (R) and index (I):
  - Relation cardinality: # tuples (**NTuples**) in *R*
  - Relation size: # pages (**NPages**) in *R*

  - Index cardinality: # distinct values (**NKeys**) in *I*
  - Index size: # pages (**INPages**) in *I*
  - Index height: # nonleaf levels (**IHeight**) of *I*
  - Index range: low/high key values (**Low**/**High**) in *I*

  - Number of distinct values in an attribute (NVal, also denoted **NKeys** for convenience)
  - Histogram for an attribute (to be seen)

Size estimation & reduction factors

```
SELECT  attribute list
FROM  relation list
WHERE  term_1 AND ... AND term_k
```

Reduction factor (RF) or Selectivity of each *term* reflects the impact of the *term* in reducing result size.

- *Assumption 1*: *uniform distribution of the values.*
- Term *col=value*: RF = *1/NKeys(I), if there is an index I on col.*
- Term *col>value*: RF = *(High(I)-value)/(High(I)-Low(I))*
- Term *col1=col2*: RF = *1/MAX(NKeys(I1), NKeys(I2))*
  - Assume R, the relation with index I1, has fewer distinct values
  - We assume each value from R has a matching value in S; in other words, {distinct values in R} is subset of {distinct values in S}.
  - Values in S are evenly distributed.
  - So each R tuple has NTuples(S)/NKeys(I2) matches, a RF of 1/NKeys(I2).

Size estimation & reduction factors

SELECT attribute list
FROM relation list
WHERE *term1* AND ... AND *termk*

Reduction factor (RF) or Selectivity of each *term*:

- *Assumption 1*: *uniform distribution of the values.*
- Term *col=value*: RF = *1/NKeys(I)*
- Term *col>value*: RF = *(High(I)-value)/(High(I)-Low(I))*
- Term *col1=col2*: RF = *1/MAX(NKeys(I1), NKeys(I2))*

Result cardinality = Max # tuples * product of all RF's.

- Max # tuples = the product of the cardinalities of relations in the FROM clause.
- *Assumption 2*: *terms are independent.*

Issue with Assumption 1

- "Uniform distribution of values": often causes highly inaccurate estimates
  - E.g., distribution of gender: male (15), female (4)
  - E.g., distribution of age:

age	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
frequency	2	3	3	1	2	1	3	8	4	2	0	1	2	4	9

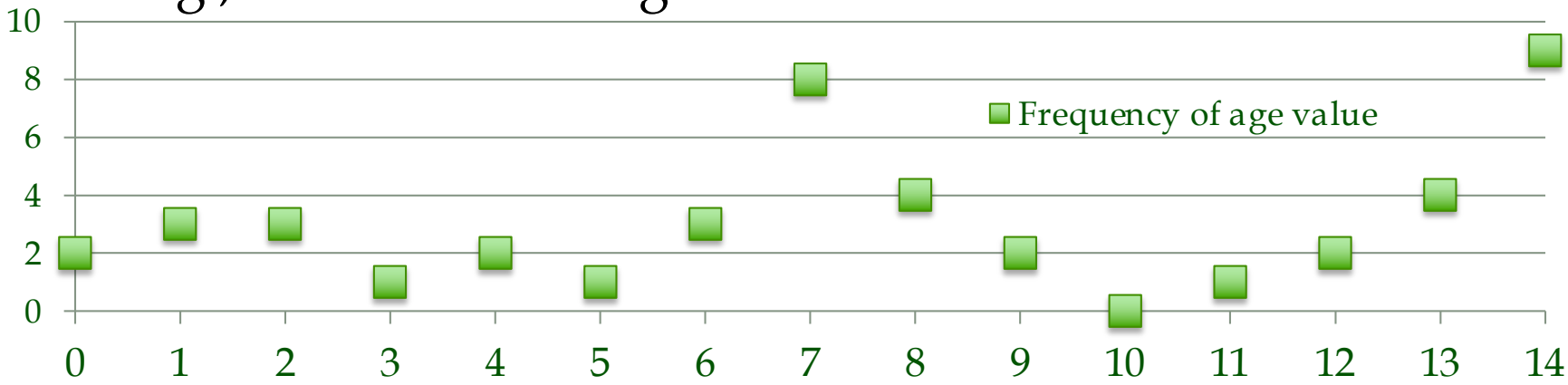
Nkeys = 15, count = 45.

Reduction factor of '*age=14*' : estimated 1/15, real 9/45=1/5

- *Histogram*: approximates a data distribution

Issue with Assumption 1

- "Uniform distribution of values": often causes highly inaccurate estimates
  - E.g., distribution of gender: male (15), female (4)
  - E.g., distribution of age:



Nkeys = 15, count = 45.

Reduction factor of '*age=14*' : estimated 1/15, real 9/45=1/5

- *Histogram*: approximates a data distribution

Equiwidth histograms

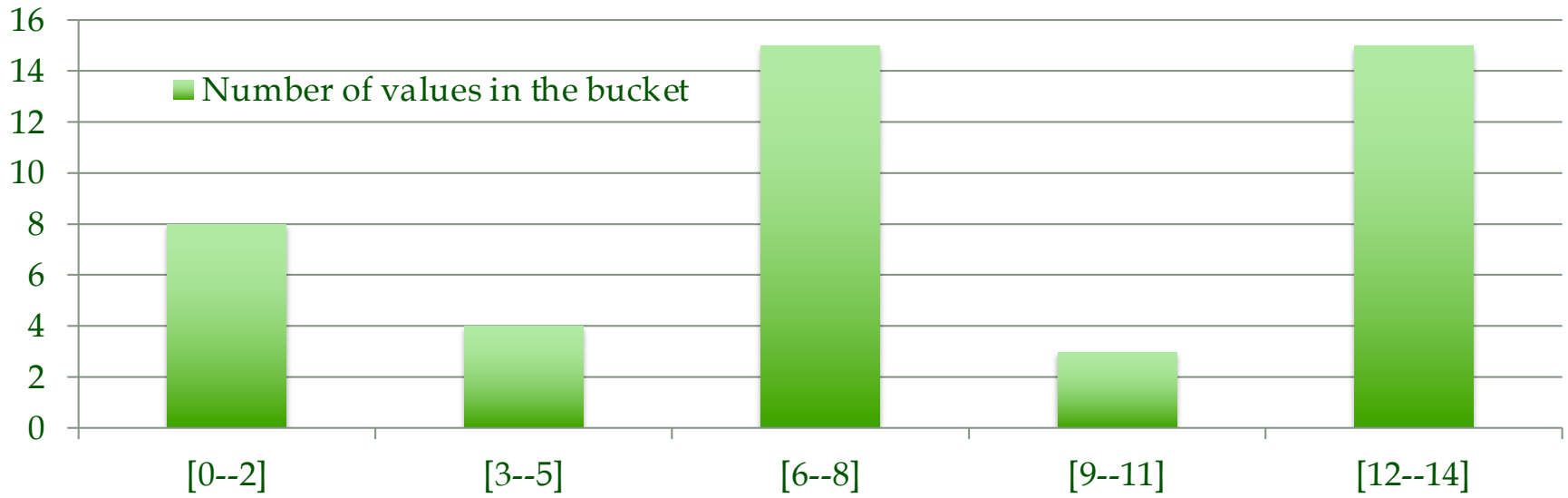
Equiwidth Histogram: buckets of equal size.

Frequency	8/3	4/3	15/3	3/3	15/3
Counts	8	4	15	3	15

Buckets	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

How many records have age=14? 15/3=5

Still not accurate for value 14 with true frequency 9

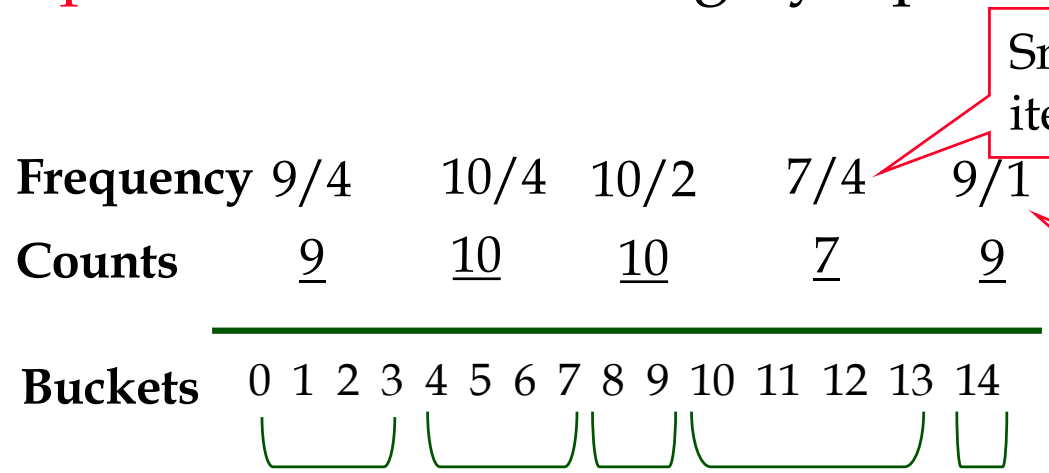


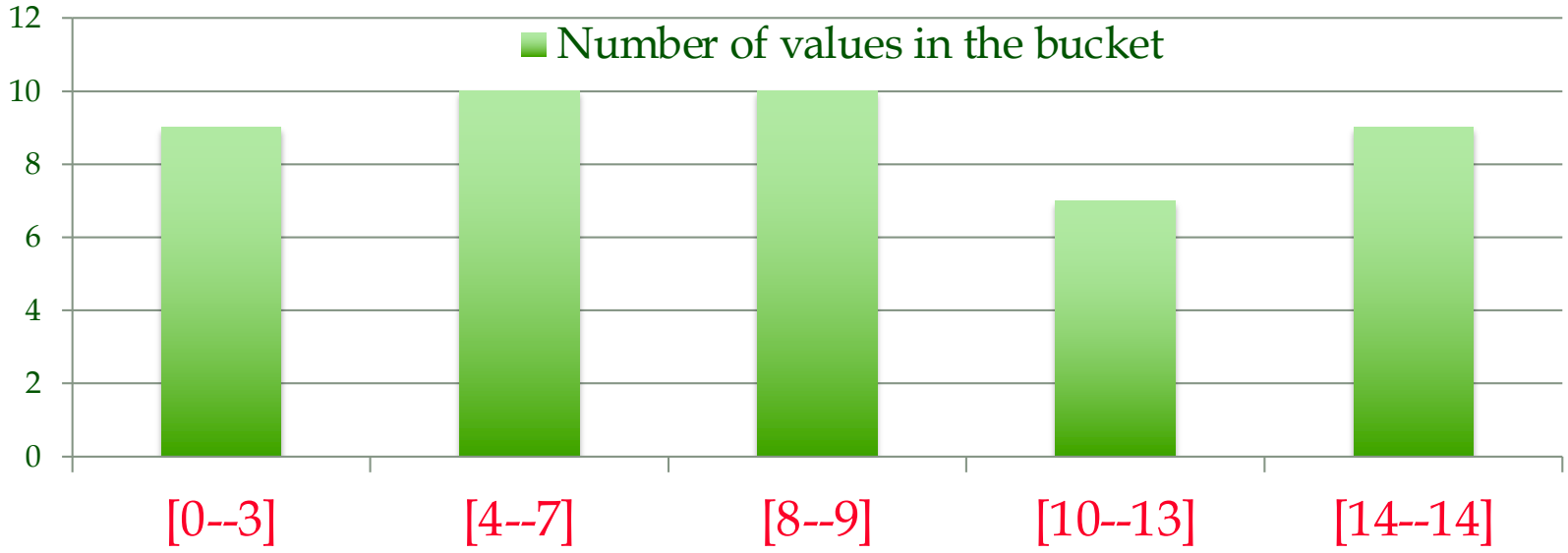
Equidepth histograms

Equidepth: buckets have roughly equal counts.

Small errors for infrequent items: tolerable.

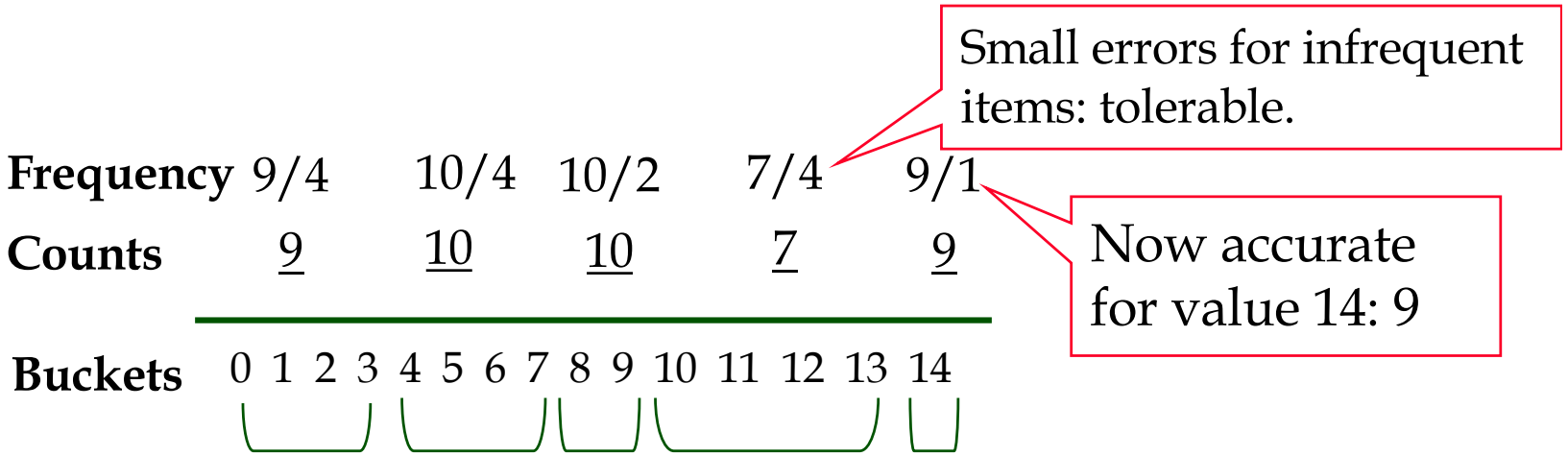
Now accurate for value 14: 9





Equidepth histograms

Equidepth: buckets have roughly equal counts.



- Favors *frequent* values.
- What if we have 100 distinct values and 20 frequent values?
  - Keep a list of most common values (MCVs) with their frequencies.
  - The histogram excludes MCVs.

https://www.postgresql.org/docs/current/planner-stats.html

Selectivity information in Postgres

The pg_stats view (easier to read) exposes fields from pg_statistics

Attr. name	**Attr. type**	**Significance**
schemaname	name	
tablename	name	
attname	boolean	
inherited	boolean	
null_frac	real	Proportion of null values
avg_width	integer	Average width in bytes
n_distinct	real	Number of distinct values
most_common_vals	anyarray	A subset of the most common values
most_common_freqs	real[]	The frequencies of mcv
histogram_bounds	anyarray	Histogram over the non-mcv
correlation	real	Correlation physical order in the store / logical value order [-1,1]

Issue with Assumption 2

- "Independence of predicates": often causes inaccurate estimates
  - E.g., Car DB: 10 makes, 100 models.
  - RF of make='honda' and model='civic'
  - If independent, 1/10 * 1/100. In practice, much higher!
- Multi-dimensional histograms
  - Maintain counts and frequency in multi-attribute space.
- Dependency-based histograms
  - Learn dependency between attributes and compute conditional probability P(model='civic' | make='honda')

https://www.postgresql.org/docs/current/planner-stats.html

Selectivity estimation experience from the field





Guy Lohman

APRIL 10, 2014

- Guy Lohman (IBM), ACM SIGMOD blog, 2014

« The root of all evil… is the estimation of cardinalities. »

What still introduces the most error in cardinality estimation is:

(a) host variables and parameter markers [parameterized queries]

(b) the selectivity of join predicates, and, even more significantly,

(c) how we **combine** selectivities to estimate the cardinality.
Example:
R(a1, a2, ..., an, compositeAttributeBuiltFrom-a1-a2...-an)
10^6 rows, no duplicates
Query 1: predicates on a1, ..., an
Query 2: identical to Q1 + redundant predicate on the composite attribute. *7 orders of magnitude slower because the optimizer thought its RF was 10^{-6} (and it was 1.0).*

Cost estimates for 1-Relation Plans

- Index I on primary key matches selection:
  - *Cost of lookup = Height(I)+1 for a B+ tree; ≈1.2 for hash index*
  - *Cost of record retrieval = 1*
- Clustered index I matching one or more selections:
  - *Cost of lookup + product of RF's of matching terms (RF-terms) * (INPages'(I)+NPages(R))*
- Non-clustered index I matching one or more selections:
  - *Cost of lookup + RF-terms * INPages'(I) + min(RF-terms * NTuples(R), NPages(R))* **demo!**
- Sequential scan of file: *NPages(R)*
- May add extra costs for GROUP BY and duplicate elimination (if the query asks for DISTINCT results)

Cost estimation for multi-relation plans

SELECT attribute list
FROM relation list
WHERE *term1* AND ... AND *termk*

- Consider a query block:
- *Reduction factor (RF)* is associated with each *term*.
- *Result cardinality* = size of cartesian product * product of all RFs.

- Multi-relation plans are built up by joining one new relation at a time.
  - Cost of join method and estimate of join cardinality gives both cost estimate and result size estimate.

Outline

- Overview of query optimization
- Enumeration of logical plans
- Enumeration of physical plans
- Estimating plan costs
- Search strategy
- Optimization of nested queries

Search strategy for 1-relation queries

- Query involves selection, projection, and aggregation.
- Enumeration of alternative plans:

1. Each *available access path* (file/index scan) is considered, the one with least estimated cost is chosen.
2. Various operations are often carried out together:
   - If an index is used for a selection, projection is done for each retrieved tuple.
   - If no GROUP BY, the resulting tuples can be *pipelined* into the aggregate computation.
   - Otherwise, hashing or sorting is needed for GROUP BY.

System R plan search

- Left-deep join plans differ in:
  - the *order* of relations,
  - the *access path* for each relation,
  - the *join method* for each join,
  - where *selections* are placed.
- Many of these plans share common prefixes, so don't enumerate all of them. This is a job for…
- Dynamic Programming: a method of solving problems that exhibit the properties of *overlapping subproblems* and *optimal substructure*
  - Overlapping subproblems: optimizing R join S join T and R join S join U share the sub-problem R join S
  - Optimal substructure: the best solution for joining R and S will be part of the best solution for joining R, S and T

Enumeration of left-deep plans

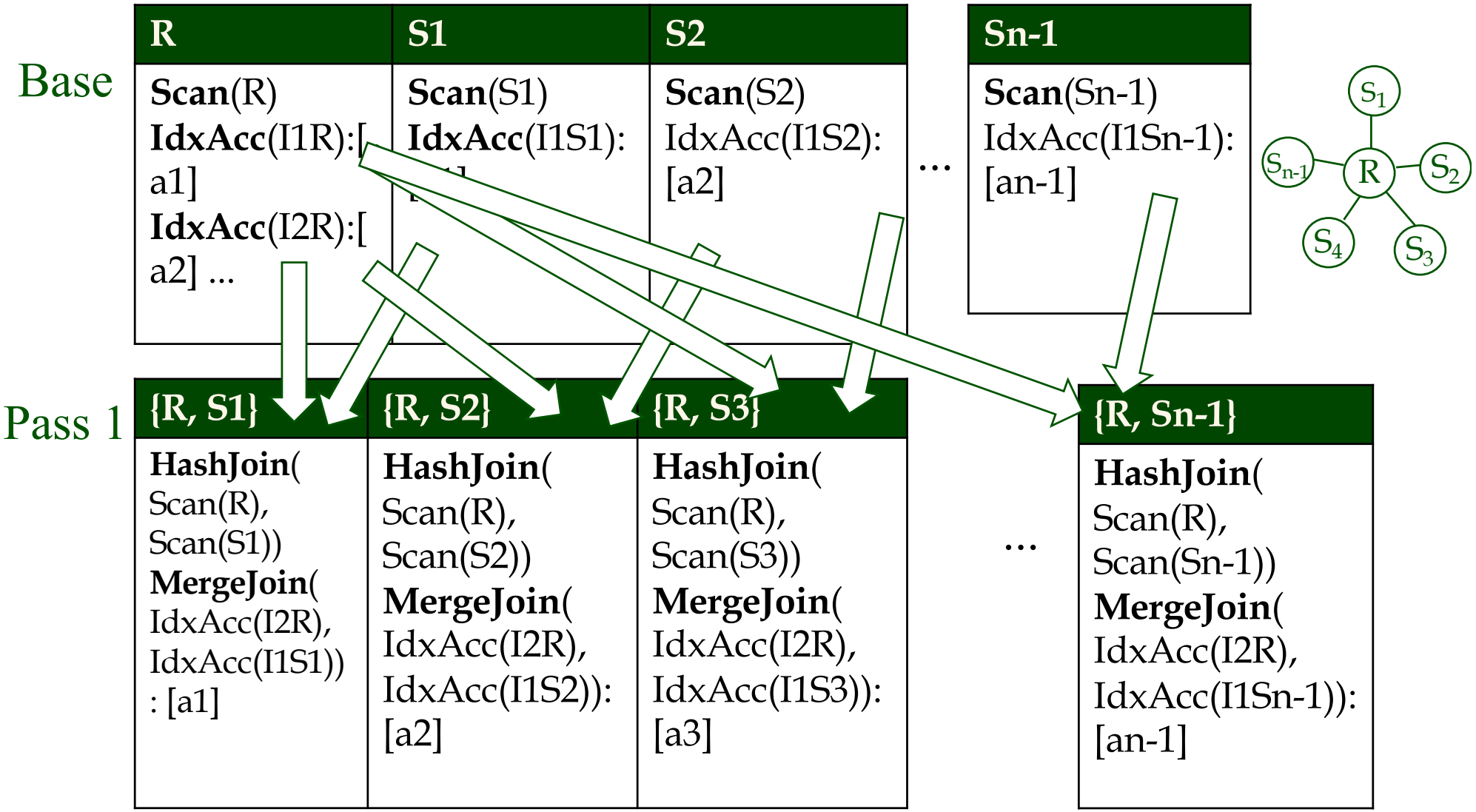
- Enumerate with N passes (if N relations are joined):
  - Base: Find best 1-relation plan for each relation.
  - Pass 1: Find best ways to join result of each 1-relation plan (as *outer*) to another relation. *(All 2-relation plans.)*
  - …
  - Pass N-1: Find best ways to join result of a (N-1)-relation plan (as *outer*) to the Nth relation. *(All N-relation plans.)*
- For each subset of relations, retain only:
  - cheapest *unordered* plan, and
  - cheapest plan for each *interesting order* (order for final output or a subsequent op. using sorting) of the tuples.

Enumeration of left-deep plans

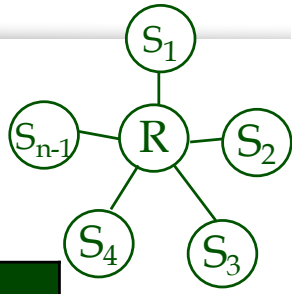
- A *k*-way (*k*<*N*) plan is not combined with an additional relation unless there is a join condition between them.
  - Do it until all predicates in WHERE have been used.
  - Avoid cartesian products if possible.

- ORDER BY, GROUP BY, aggregates etc. handled as a final step, using an 'interestingly ordered' plan, or an additional sorting or hashing.

Plan search with Dynamic Programming



Plan search with Dynamic Programming



Base

R	S1	S2	Sn-1
...	..	..	...

Pass 1

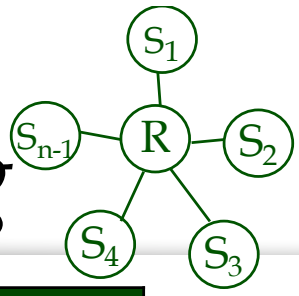
{R, S1}	{R, S2}	{R, S3}	{R, Sn-1}
HashJoin(Scan(R), Scan(S1)) **MergeJoin**(IdxAcc(I2R) IdxAcc(I1S): [a1]	**HashJoin**(Scan(R), S (S2)) **MergeJo** (IdxAc 2R), IdxA (I1S2)): [a1]	**HashJoin**(Scan(R), Scan(S3)) **MergeJoin**(IdxAcc(I2R), IdxAcc(I1S3)): [a2]	**HashJoin**(Scan(R), Scan(Sn-1)) **MergeJoin**(IdxAcc(I2R), IdxAcc(I1Sn-1)): [an-1]

...

Pass 2

{R, S1, 2}	{R, S2, S3}	{R, S1, S3}	{R, Sn-2, Sn-1}
HashJoin(**HashJoin**(Scan(R), Scan(S1)), Scan(S2)) **MergeJoin**(...) ...	**HashJoin**(S2, HashJoin(Scan(R), Scan(S3))) **MergeJoin**(...):...	**HashJoin**(HashJoin(Scan(R), Scan(S3)), Scan(S1)) ... **MergeJoin**(...):...	HashJoin(HashJoin(...), ...), MergeJoin(...):...

Plan search with Dyn. Programming



Base

R	S1	S2	…	Sn-1
…	…	…		…

Pass 1

{R, S1}	{R, S2}	{R, S3}	…	{R, Sn-1}
…	…	…		…

Pass 2

{R, S1, S2}	{R, S2, S3}	{R, S1, S3}	…	{R, Sn-2, Sn-1}
…	…	…		…

…

Pass n-2

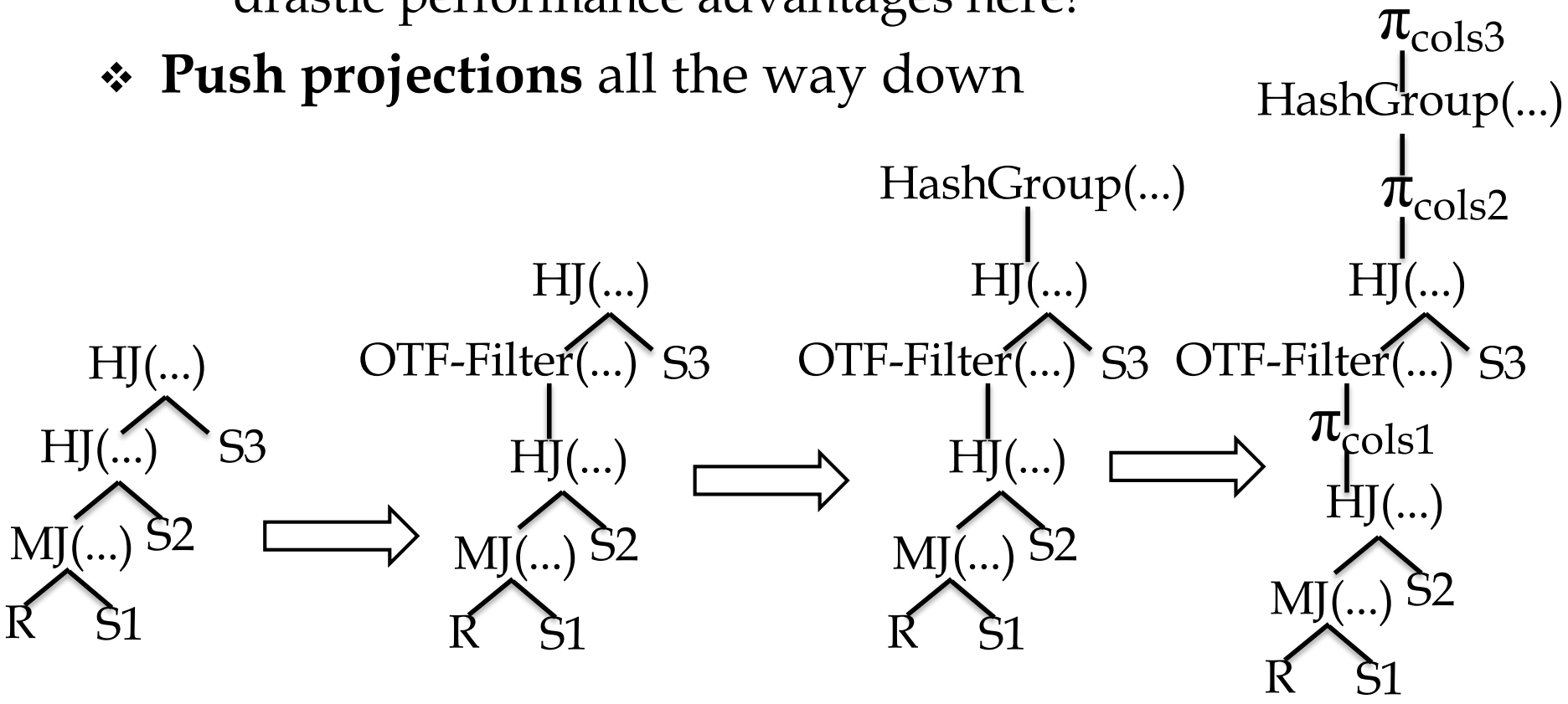
{All but S1}	{All but S2}	{All but S3}	…	{All but Sn-1}
…	…	…		…

Pass n-1

{R, S1, S2, … Sn-1}
1 unordered plan and 1 plan for each interesting order

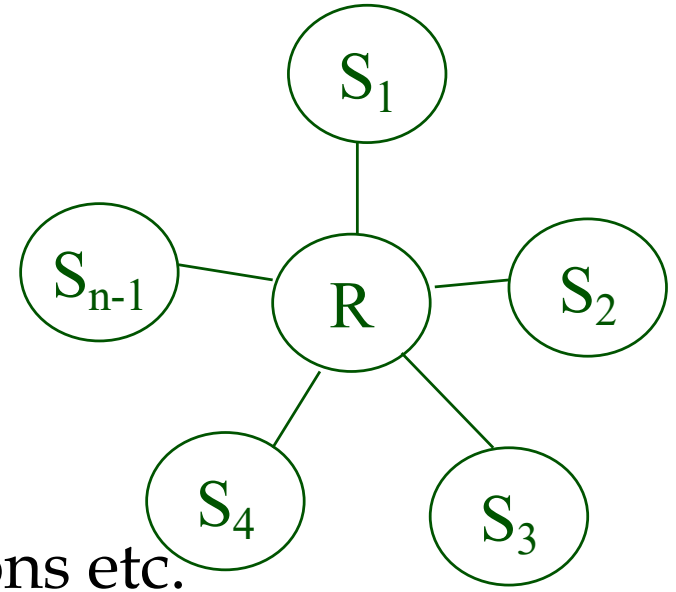
Final query optimization steps

- **Apply remaining selections**, if any
  - Push them as far down as possible
- **Apply Group-By, Having, Order** by if any
  - Some interesting orders in the full join plan may bring drastic performance advantages here!
- **Push projections** all the way down



Complexity of plan enumeration

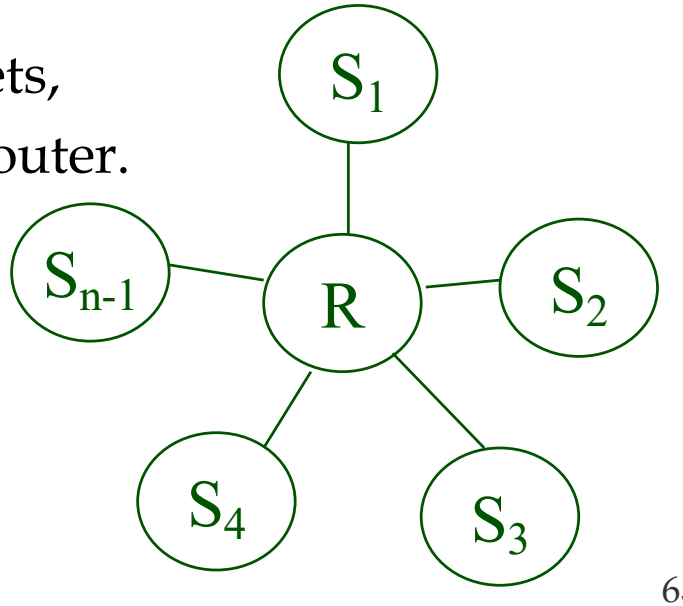
- Enumeration of all left-deep plans for an n-way join: $O(n!)$
- System R more efficient: consider a star join graph
  - $R.a_1 = S_1.a_1$
  - $R.a_2 = S_2.a_2$
  - …
  - $R.a_{n-1} = S_{n-1}.a_{n-1}$



- The query may also involve selections etc.
- Other query "shapes": line or chain, clique (rare)

Complexity with a "star graph" query

- Total number of plans considered:
  - Pass 2: (n-1 choose 1) 2-relation subsets,
    for each subset, pick one as the outer reln in the join
    (best plan for the inner has been chosen in the previous pass).
  - Pass 3: (n-1 choose 2) 3-relation subsets,
    for each subset, pick one as the outer.
  - …
  - Pass n: (n-1 choose n-1) n-relation subsets,
    for each subset, pick one as the outer.
  - Total number of plans = $O(n \cdot 2^{n-1})$
- Maximum number of plans stored in a pass?

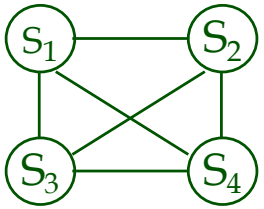


Influence of the query shape

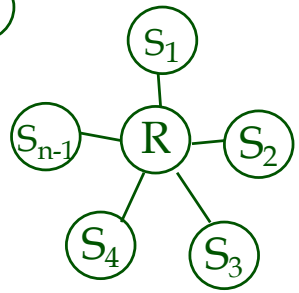
- Among queries without cartesian products:
  - **Line** (chain) queries *minimize* the number of plans (→ simplify optimization!) as a reln. only joins with previous and next reln.



  - **Clique** queries *maximize* the number of plans as each relation joins with every other relation
    - Queries of the form R.a=S.a=T.a=...



  - **Star** queries very frequent in data warehousing
    - Central table: "facts"
    - Other tables joining with it: "dimensions"



  - **Real** queries are a mix of various shapes

Query plans in Postgres

- To obtain query plans use EXPLAIN: https://www.postgresql.org/docs/current/sql-explain.html
- EXPLAIN ANALYZE also runs the query, measures perf.
- Usage in the interactive client:
  - EXPLAIN SELECT ... → plan → you can copy-paste in https://explain.dalibo.com
- Usage outside of the interactive client:
  - Write in query.sql: EXPLAIN SELECT...
  - Call: psql -XqAt -f query.sql > analyze.out

Outline

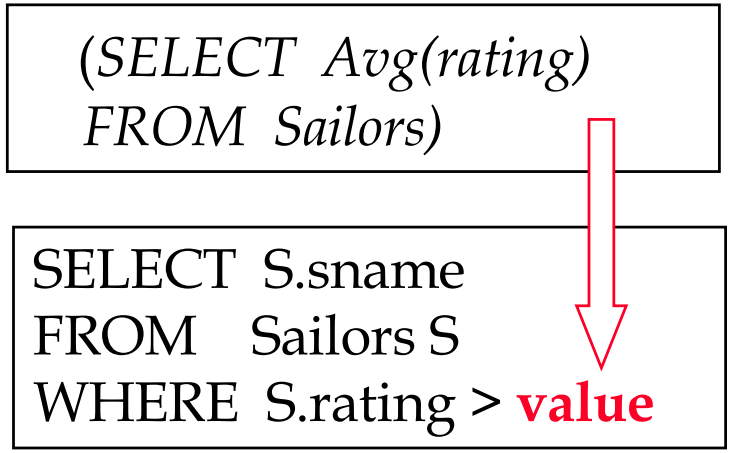
- Overview of query optimization
- Enumeration of logical plans
- Enumeration of physical plans
- Estimating plan costs
- Search strategy
- Optimization of nested queries

Nested queries with no correlation

- *Nested query*: appears as an operand of a predicate in WHERE.

- *Nested query with no correlation*: does not contain a reference to the tuple from the outer.
  - A nested query needs to be evaluated *only once*.
  - The optimizer arranges it to be evaluated before the top level query.

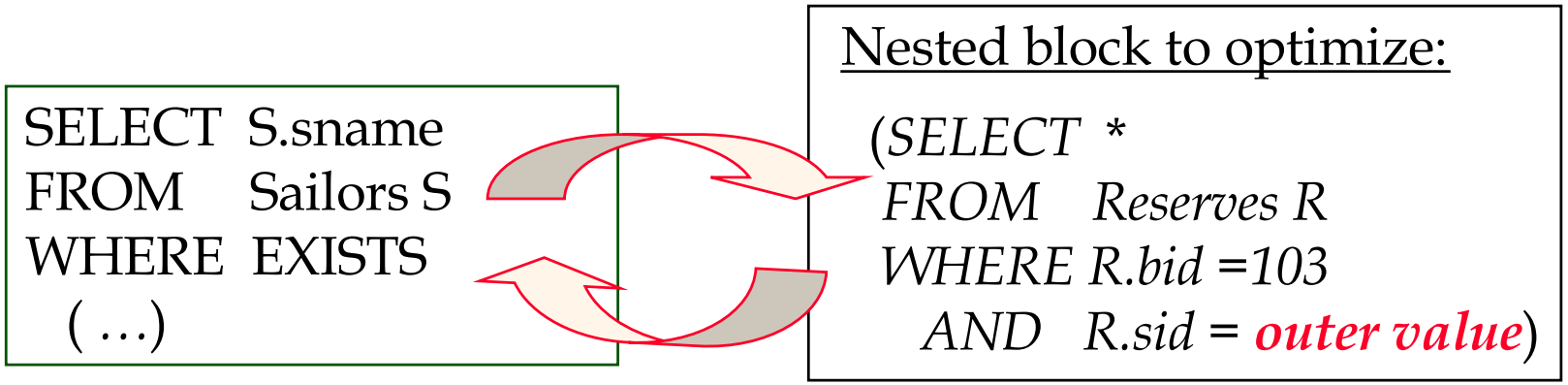
```
SELECT  S.sname
FROM    Sailors S
WHERE  S.rating >
   (SELECT  Avg(rating)
    FROM  Sailors)
```



Nested Queries With Correlation

- Nested query with correlation: contains a reference to a tuple from the outer.
- Naive approach: nested block is optimized independently, with the value of outer tuple considered as a constant.
  - The nested block is evaluated for each outer value.

```
SELECT  S.sname
FROM    Sailors S
WHERE  EXISTS
  (SELECT  *
   FROM    Reserves R
   WHERE  R.bid=103
    AND     R.sid=S.sid)
```



Query Decorrelation

- The equivalent, non-nested version of the query is optimized better, usually with 1 extra join

- *Query decorrelation* flattens the query with nested blocks to an equivalent single block.

```
SELECT  S.sname
FROM    Sailors S
WHERE  EXISTS
  (SELECT  *
   FROM     Reserves R
   WHERE  R.bid=103
    AND      R.sid=S.sid)
```

Equivalent non-nested query:

```
SELECT  S.sname
FROM    Sailors S, Reserves R
WHERE  S.sid=R.sid
    AND  R.bid=103
```

Query Decorrelation (Contd.)

- *Guideline: Use only one "query block", if possible.*

```
SELECT DISTINCT *
FROM   Sailors S
WHERE S.sname IN
        (SELECT Y.sname
         FROM YoungSailors Y)
```

=

```
SELECT DISTINCT S.*
FROM   Sailors S,
       YoungSailors Y
WHERE S.sname = Y.sname
```

- *Not always possible ...*

```
SELECT *
FROM   Sailors S
WHERE S.sname IN
       (SELECT [DISTINCT] Y.sname
        FROM YoungSailors Y)
```

≠

```
SELECT S.*
FROM   Sailors S,
       YoungSailors Y
WHERE S.sname = Y.sname
```

Questions



Summary

- Query optimization is an **important** task in relational DBMS.
- Must understand optimization in order to understand the performance impact of a given database design (relations, indexes) on a workload (set of queries).
- Query optimization by the database server:
  - Bringing it to a "simpler" canonical form
    - Decorrelation; normalization of logical conditions
  - Consider **a set of alternative plans**.
    - Prune search space; typically, left-deep plans only.
  - Estimate **cost** of each plan that is considered.
    - Estimate size of result and cost for each plan node.
    - *Key issues*: Statistics, indexes, operator implementations.

Summary (Contd.)

- Single-relation queries:
  - All access paths considered, cheapest is chosen.
  - *Issues*: Selections that *match* index, whether index key has all needed fields and/or provides tuples in a desired order.
- Multiple-relation queries:
  - All single-relation plans are first enumerated.
    - Selections/projections considered as early as possible.
  - Next, for each 1-relation plan, all ways of joining another relation (as inner) are considered.
  - Next, for each 2-relation plan that is 'retained', all ways of joining another relation (as inner) are considered, etc.
  - *At each level, for each subset of relations, only best plan for each interesting order of tuples is 'retained'.*